

# Redes Neurais e Deep Learning

SGD

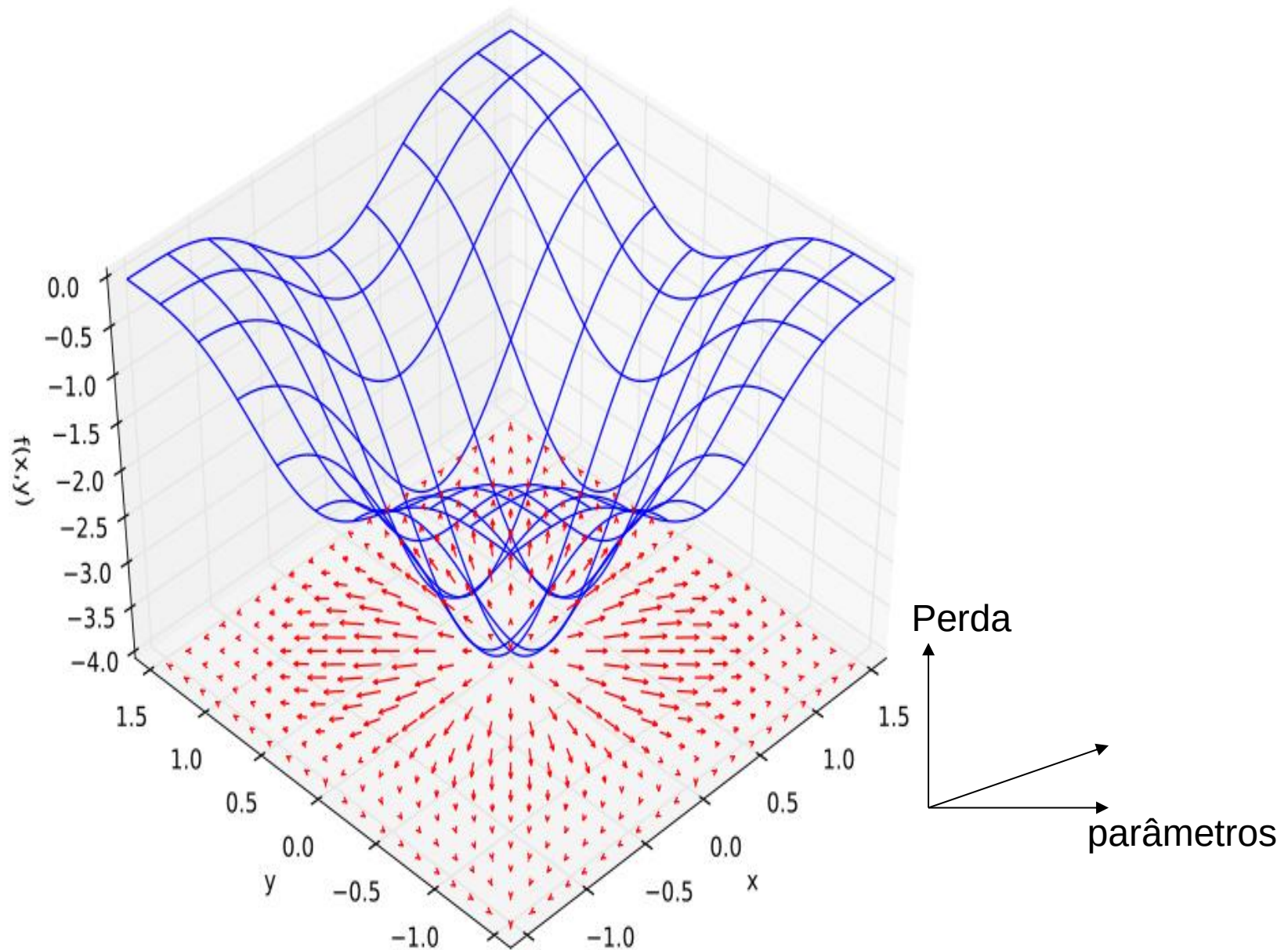
*STOCHASTIC GRADIENT DESCENT*

---

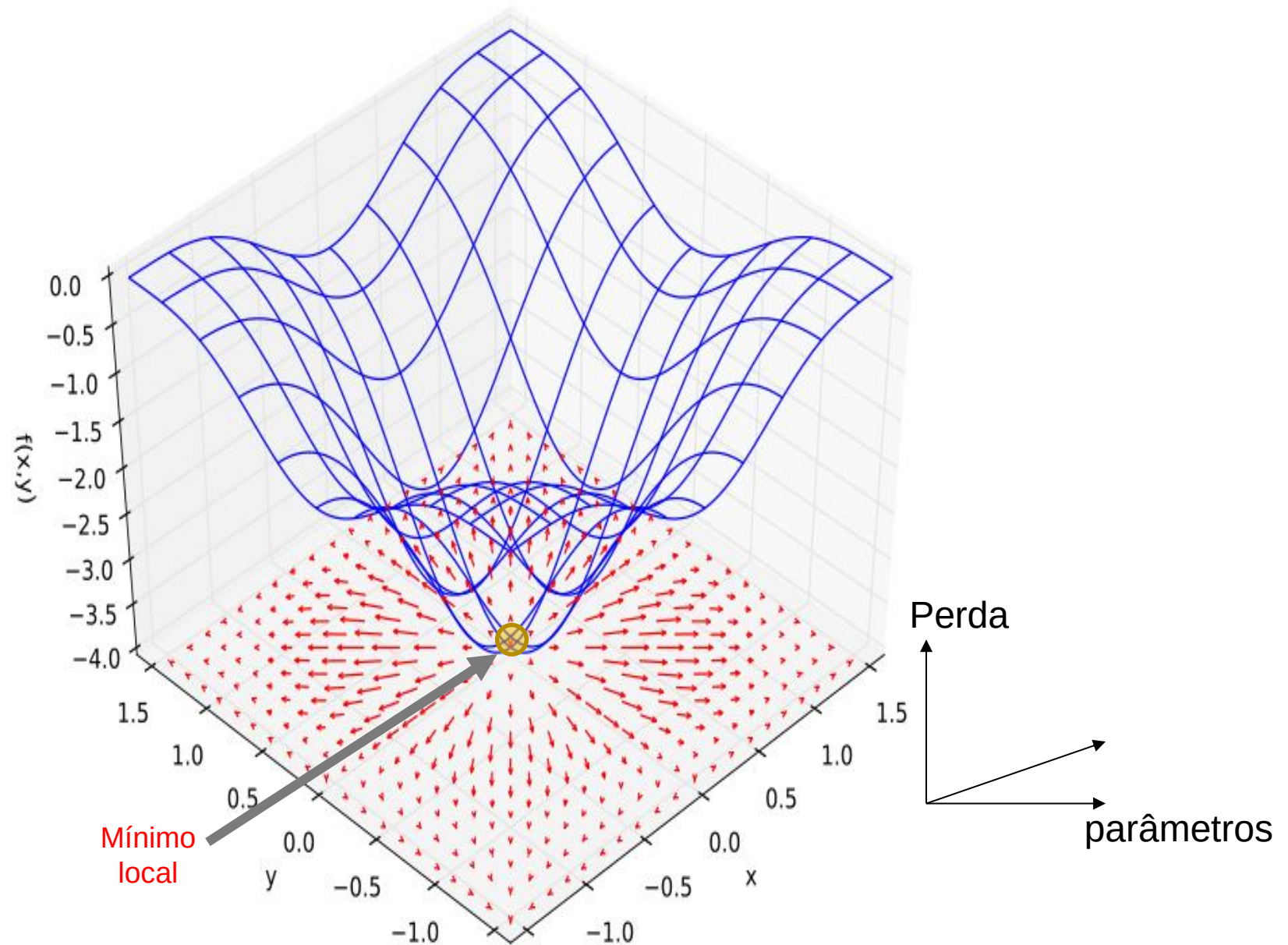
Zenilton K. G. Patrocínio Jr

[zenilton@pucminas.br](mailto:zenilton@pucminas.br)

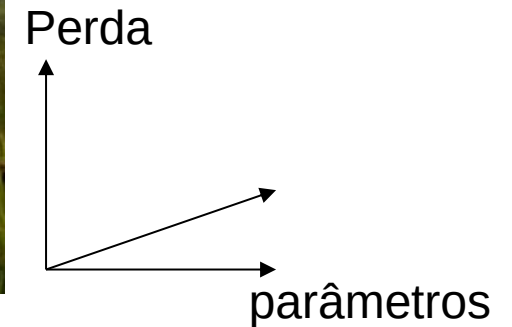
# Otimização da Função de Perda



# Otimização da Função de Perda

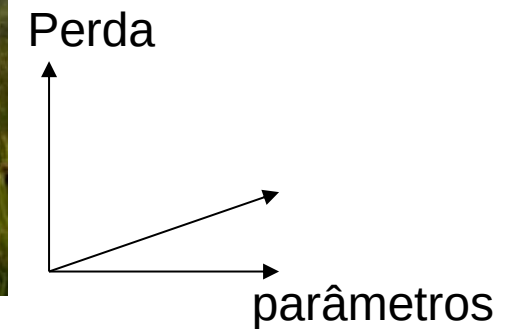


# Otimização da Função de Perda





# Método 1 – Busca Randômica



# Método 1 – Busca Randômica

Talvez seja uma ideia muito ruim... mas é simples!

```
# assume X_train is the data where each column is an example (e.g. 3073 x 50,000)
# assume Y_train are the labels (e.g. 1D array of 50,000)
# assume the function L evaluates the loss function

bestloss = float("inf") # Python assigns the highest possible float value
for num in xrange(1000):
    W = np.random.randn(10, 3073) * 0.0001 # generate random parameters
    loss = L(X_train, Y_train, W) # get the loss over the entire training set
    if loss < bestloss: # keep track of the best solution
        bestloss = loss
        bestW = W
    print 'in attempt %d the loss was %f, best %f' % (num, loss, bestloss)

# prints:
# in attempt 0 the loss was 9.401632, best 9.401632
# in attempt 1 the loss was 8.959668, best 8.959668
# in attempt 2 the loss was 9.044034, best 8.959668
# in attempt 3 the loss was 9.278948, best 8.959668
# in attempt 4 the loss was 8.857370, best 8.857370
# in attempt 5 the loss was 8.943151, best 8.857370
# in attempt 6 the loss was 8.605604, best 8.605604
# ... (truncated: continues for 1000 lines)
```

# Método 1 – Busca Randômica

Quão bem se comporta essa abordagem sobre o conjunto de teste...

```
# Assume X_test is [3073 x 10000], Y_test [10000 x 1]  
scores = Wbest.dot(Xte_cols) # 10 x 10000, the class scores for all test examples  
# find the index with max score in each column (the predicted class)  
Yte_predict = np.argmax(scores, axis = 0)  
# and calculate accuracy (fraction of predictions that are correct)  
np.mean(Yte_predict == Yte)  
# returns 0.1555
```

15,5% acurácia! Não é tão ruim!

Porém, estado da arte é ~95%

# Método 1 – Busca Randômica

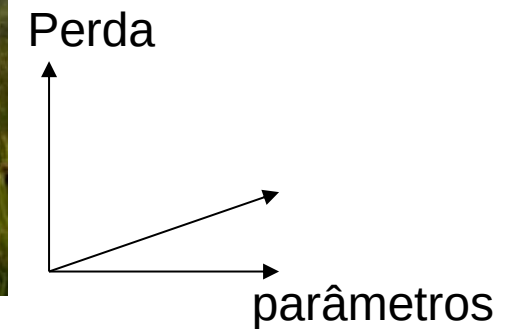
Toma-se vários passos aleatórios e mede-se a perda. Em seguida, direciona-se para o valor mais baixo encontrado

- Não é totalmente “sem sentido”. Esse é um tipo de otimização “sem gradiente”. Faz sentido se você não puder calcular gradientes por algum motivo
- Uma versão um pouco mais inteligente calcula a média dos pontos aleatórios ponderados pela perda (pontos de menor perda ganham maior peso). Isso se aproxima do gradiente

Mas é menos eficiente que o método do gradiente quando os gradientes estão disponíveis



# Método 2 – Seguindo a Inclinação



# Método 2 – Seguindo a Inclinação

Em uma dimensão, a derivada de uma função é dada por

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Em múltiplas dimensões, o **gradiente** representa o vetor de derivadas parciais

# Método 2 – Seguindo a Inclinação

Em uma dimensão, a derivada de uma função é dada por

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Em múltiplas dimensões, o **gradiente** representa o vetor de derivadas parciais

# Método 2 – Seguindo a Inclinação

Em uma dimensão, a derivada de uma função é dada por

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Em múltiplas dimensões, o **gradiente** representa o vetor de derivadas parciais



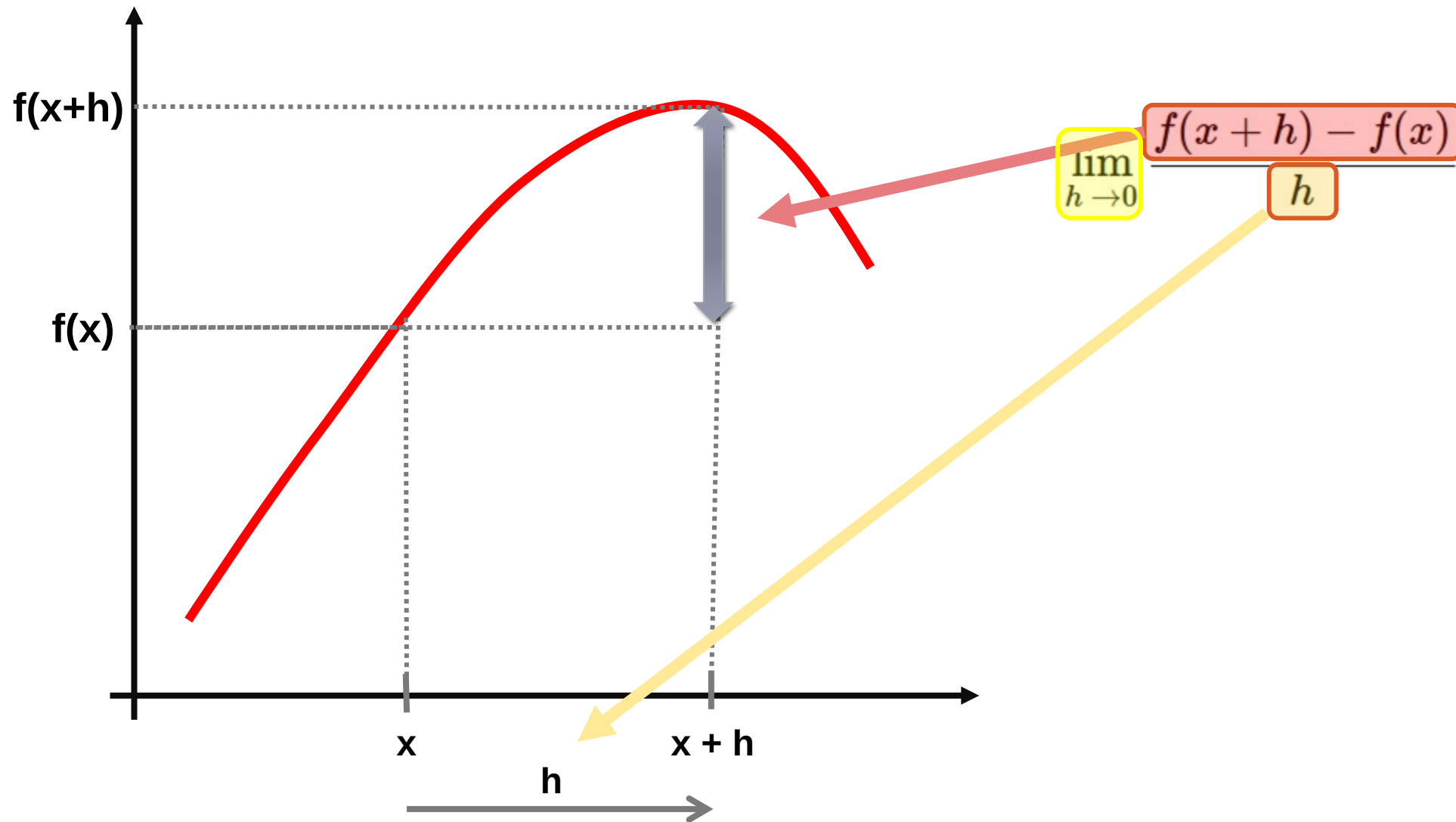
# Método 2 – Seguindo a Inclinação

Em uma dimensão, a derivada de uma função é dada por

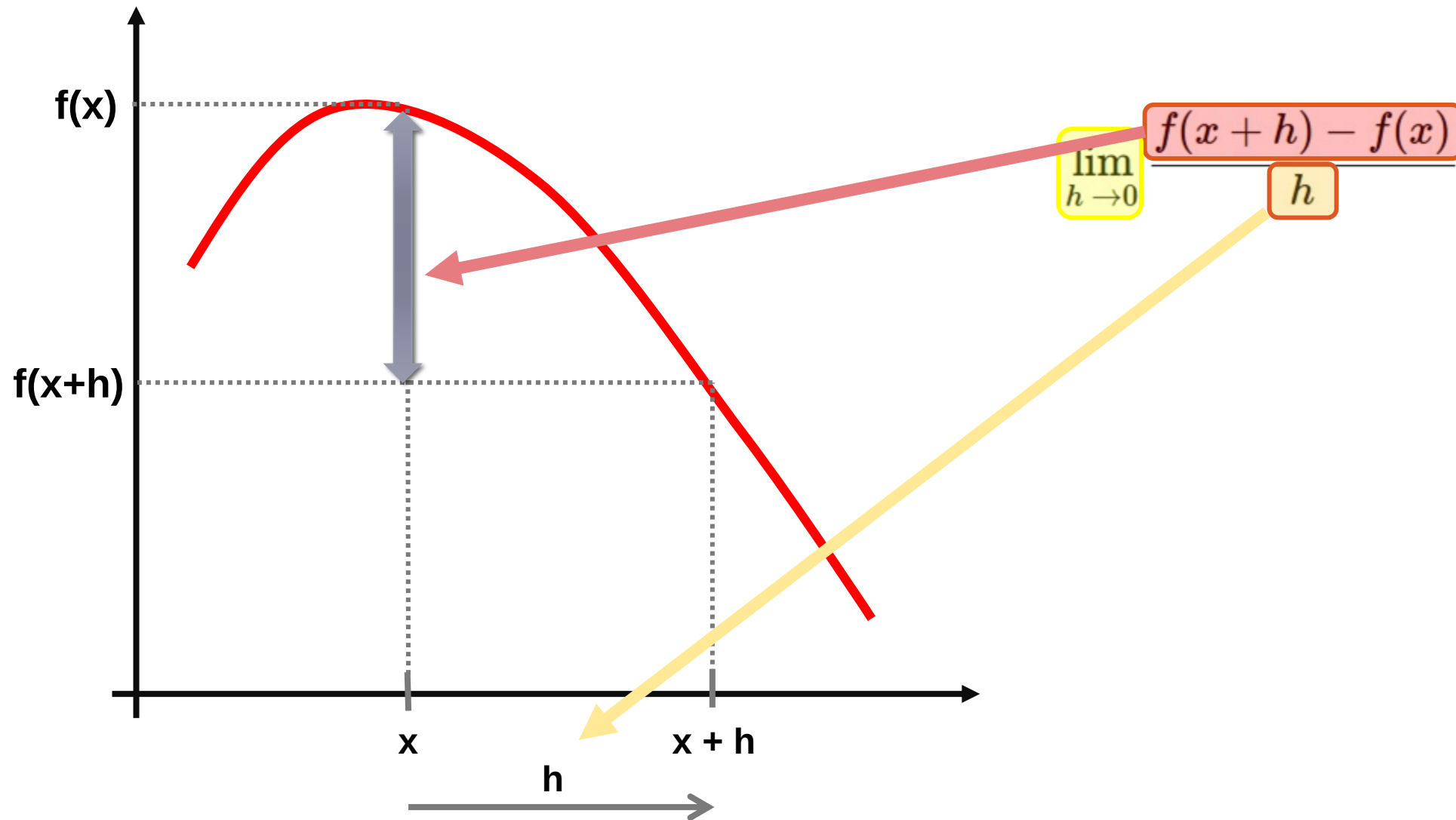
$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Em múltiplas dimensões, o **gradiente** representa o vetor de derivadas parciais

# Método 2 – Seguindo a Inclinação



# Método 2 – Seguindo a Inclinação



# Método 2 – Seguindo a Inclinação

**W corrente:**

[0,34;  
-1,11;  
0,78;  
0,12;  
0,55;  
2,81;  
-3,1;  
-1,5;  
0,33;...]

**perda 1,25347**

**gradiente dW:**

[?;  
?;  
?;  
?;  
?;  
?;  
?;  
?;  
?;  
?;...]

# Método 2 – Seguindo a Inclinação

**W corrente:**

[0,34;  
-1,11;  
0,78;  
0,12;  
0,55;  
2,81;  
-3,1;  
-1,5;  
0,33;...]

**perda 1,25347**

**W + h (1ª dim):**

[0,34 + **0,0001**;  
-1,11;  
0,78;  
0,12;  
0,55;  
2,81;  
-3,1;  
-1,5;  
0,33;...]

**perda 1,25322**

**gradiente dW:**

[?;  
?;  
?;  
?;  
?;  
?;  
?;  
?;  
?;...]



# Método 2 – Seguindo a Inclinação

**W corrente:**

[0,34;  
-1,11;  
0,78;  
0,12;  
0,55;  
2,81;  
-3,1;  
-1,5;  
0,33;...]

**perda 1,25347**

**W + h (1ª dim):**

[0,34 + **0,0001**;  
-1,11;  
0,78;  
0,12;  
0,55;  
2,81;  
-3,1;  
-1,5;  
0,33;...]

**perda 1,25322**

**gradiente dW:**

**[-2,5;**  
?  
?

$$(1,25322 - 1,25347)/0,0001 = -2,5$$

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

?;  
?;...]

# Método 2 – Seguindo a Inclinação

**W corrente:**

[0,34;  
-1,11;  
0,78;  
0,12;  
0,55;  
2,81;  
-3,1;  
-1,5;  
0,33;...]

**perda 1,25347**

**W + h (2ª dim):**

[0,34;  
-1,11 + **0,0001**;  
0,78;  
0,12;  
0,55;  
2,81;  
-3,1;  
-1,5;  
0,33;...]

**perda 1,25353**

**gradiente dW:**

[-2,5;  
?;  
?;  
?;  
?;  
?;  
?;  
?;  
?;...]

# Método 2 – Seguindo a Inclinação

**W corrente:**

[0,34;  
-1,11;  
0,78;  
0,12;  
0,55;  
2,81;  
-3,1;  
-1,5;  
0,33;...]

**perda 1,25347**

**W + h (2ª dim):**

[0,34;  
-1,11 + **0,0001**;  
0,78;  
0,12;  
0,55;  
2,81;  
-3,1;  
-1,5;  
0,33;...]

**perda 1,25353**

**gradiente dW:**

[-2,5;  
**0,6**;  
?;  
?;

$$(1,25353 - 1,25347)/0,0001 = 0,6$$

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

?;...]

# Método 2 – Seguindo a Inclinação

**W corrente:**

[0,34;  
-1,11;  
0,78;  
0,12;  
0,55;  
2,81;  
-3,1;  
-1,5;  
0,33;...]

**perda 1,25347**

**W + h (3ª dim):**

[0,34;  
-1,11;  
0,78 + **0,0001**;  
0,12;  
0,55;  
2,81;  
-3,1;  
-1,5;  
0,33;...]

**perda 1,25347**

**gradiente dW:**

[-2,5;  
0,6;  
?;  
?;  
?;  
?;  
?;  
?;  
?;...]

# Método 2 – Seguindo a Inclinação

**W corrente:**

[0,34;  
-1,11;  
0,78;  
0,12;  
0,55;  
2,81;  
-3,1;  
-1,5;  
0,33;...]

**perda 1,25347**

**W + h (3ª dim):**

[0,34;  
-1,11;  
0,78 + **0,0001**;  
0,12;  
0,55;  
2,81;  
-3,1;  
-1,5;  
0,33;...]

**perda 1,25347**

**gradiente dW:**

[-2,5;  
0,6;  
**0**;  
?;

$$(1,25347 - 1,25347)/0,0001 = 0$$

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

?;...]

# Método 2 – Seguindo a Inclinação

## Avaliação numérica do gradiente

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

```
def eval_numerical_gradient(f, x):  
    """  
    a naive implementation of numerical gradient of f at x  
    - f should be a function that takes a single argument  
    - x is the point (numpy array) to evaluate the gradient at  
    """  
  
    fx = f(x) # evaluate function value at original point  
    grad = np.zeros(x.shape)  
    h = 0.00001  
  
    # iterate over all indexes in x  
    it = np.nditer(x, flags=['multi_index'], op_flags=['readwrite'])  
    while not it.finished:  
  
        # evaluate function at x+h  
        ix = it.multi_index  
        old_value = x[ix]  
        x[ix] = old_value + h # increment by h  
        fxh = f(x) # evaluate f(x + h)  
        x[ix] = old_value # restore to previous value (very important!)  
  
        # compute the partial derivative  
        grad[ix] = (fxh - fx) / h # the slope  
        it.iternext() # step to next dimension  
  
    return grad
```

# Método 2 – Seguindo a Inclinação

## Avaliação numérica do gradiente

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

- **Aproximada**
- **Muito lenta**

```
def eval_numerical_gradient(f, x):  
    """  
    a naive implementation of numerical gradient of f at x  
    - f should be a function that takes a single argument  
    - x is the point (numpy array) to evaluate the gradient at  
    """  
  
    fx = f(x) # evaluate function value at original point  
    grad = np.zeros(x.shape)  
    h = 0.00001  
  
    # iterate over all indexes in x  
    it = np.nditer(x, flags=['multi_index'], op_flags=['readwrite'])  
    while not it.finished:  
  
        # evaluate function at x+h  
        ix = it.multi_index  
        old_value = x[ix]  
        x[ix] = old_value + h # increment by h  
        fxh = f(x) # evaluate f(x + h)  
        x[ix] = old_value # restore to previous value (very important!)  
  
        # compute the partial derivative  
        grad[ix] = (fxh - fx) / h # the slope  
        it.iternext() # step to next dimension  
  
    return grad
```



# Método 2 – Seguindo a Inclinação

Uma abordagem mais promissora!

A perda é apenas uma função de  $W$ , por exemplo:

$$L = \frac{1}{N} \sum_{i=1}^N L_i + \sum_k W_k^2$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$s = f(x; W) = Wx$$

Deseja-se

$$\nabla_W L = \dots$$

# Método 2 – Seguindo a Inclinação

Uma abordagem mais promissora

A perda é apenas uma função de  $W$ , por exemplo:

$$L = \frac{1}{N} \sum_{i=1}^N L_i + \sum_k W_k^2$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$s = f(x; W) = Wx$$

Deseja-se

$$\nabla_W L = \dots$$

**Solução**  
**Cálculo Diferencial**

Newton



Leibniz



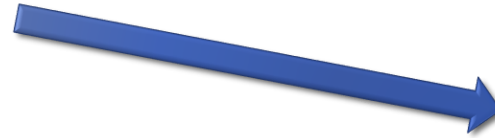
# Método 2 – Seguindo a Inclinação

**W corrente:**

[0,34;  
-1,11;  
0,78;  
0,12;  
0,55;  
2,81;  
-3,1;  
-1,5;  
0,33;...]

**perda 1,25347**

$dW = \dots$   
(alguma função dos  
dados e de W)



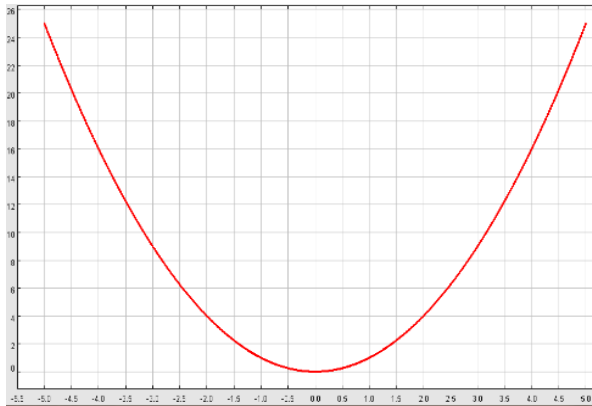
**gradiente dW:**

[-2,5;  
0,6;  
0;  
0,2;  
0,7;  
-0,5;  
1,1;  
1,3;  
-2,1;...]

# Funções de Perda

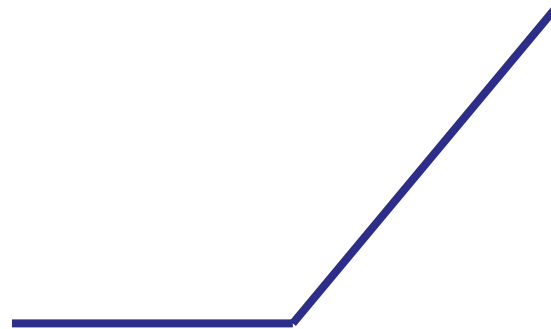
Perda Quadrática

$$L = (y_i - f(x_i))^2$$



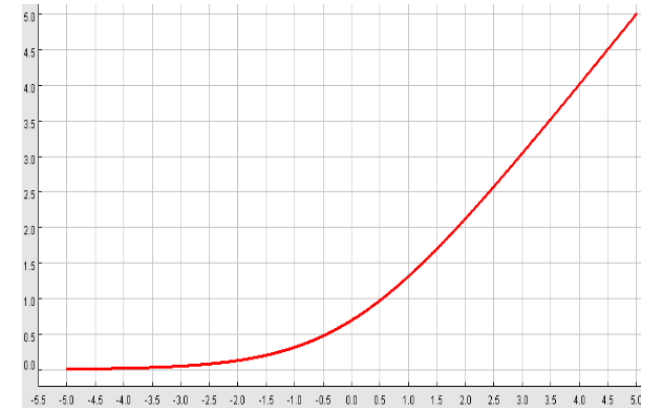
Perda de Articulação

$$y_i \in \{-1, 1\}$$
$$L = \max(0, 1 - y_i f(x_i))$$



Perda de Entropia Cruzada

$$y_i \in \{-1, 1\}$$
$$L = \log(1 + \exp(-y_i f(x_i)))$$



Todas essas três funções de perda possuem derivadas “bem comportadas”

Como  $f(x) = w^T x$  é uma função linear dos pesos  $W$ , pode-se derivar a perda em relação aos pesos

# Método 2 – Seguindo a Inclinação

- Gradiente Numérico: aproximado, lento, fácil de codificar
- Gradiente Analítico : exato, rápido, a prova de erros

**Na prática:** Sempre se usa gradiente analítico, mas ocasionalmente verifica-se a implementação por meio de gradiente numérico.

⇒ Isto é chamado de **verificação de gradiente**

# Gradiente da Função de Perda

O gradiente  $\nabla_W L(W)$  representa o vetor de derivadas parciais

$$\nabla_W L(W) = \left[ \frac{\partial L}{\partial W_{11}}, \frac{\partial L}{\partial W_{12}}, \dots, \frac{\partial L}{\partial W_{21}}, \frac{\partial L}{\partial W_{22}}, \dots \right]^T$$

# Gradiente da Função de Perda

O gradiente  $\nabla_W L(W)$  representa o vetor de derivadas parciais

$$\nabla_W L(W) = \left[ \frac{\partial L}{\partial W_{11}}, \frac{\partial L}{\partial W_{12}}, \dots, \frac{\partial L}{\partial W_{21}}, \frac{\partial L}{\partial W_{22}}, \dots \right]^T$$

em que, por exemplo,  $\frac{\partial L}{\partial W_{11}}$  mede o quão rápido varia a perda  $L$  em relação a uma variação do coeficiente  $W_{11}$  da matriz  $W$

# Gradiente da Função de Perda

O gradiente  $\nabla_W L(W)$  representa o vetor de derivadas parciais

$$\nabla_W L(W) = \left[ \frac{\partial L}{\partial W_{11}}, \frac{\partial L}{\partial W_{12}}, \dots, \frac{\partial L}{\partial W_{21}}, \frac{\partial L}{\partial W_{22}}, \dots \right]^T$$

em que, por exemplo,  $\frac{\partial L}{\partial W_{11}}$  mede o quão rápido varia a perda  $L$  em relação a uma variação do coeficiente  $W_{11}$  da matriz  $W$

Neste caso, para se minimizar o valor do risco empírico é necessário se igualar a zero o gradiente de  $L$  em relação à matriz  $W$ , isto é

$$\nabla_W L(W) = 0$$

considerando, assim, que  $L$  é uma função da matriz  $W$



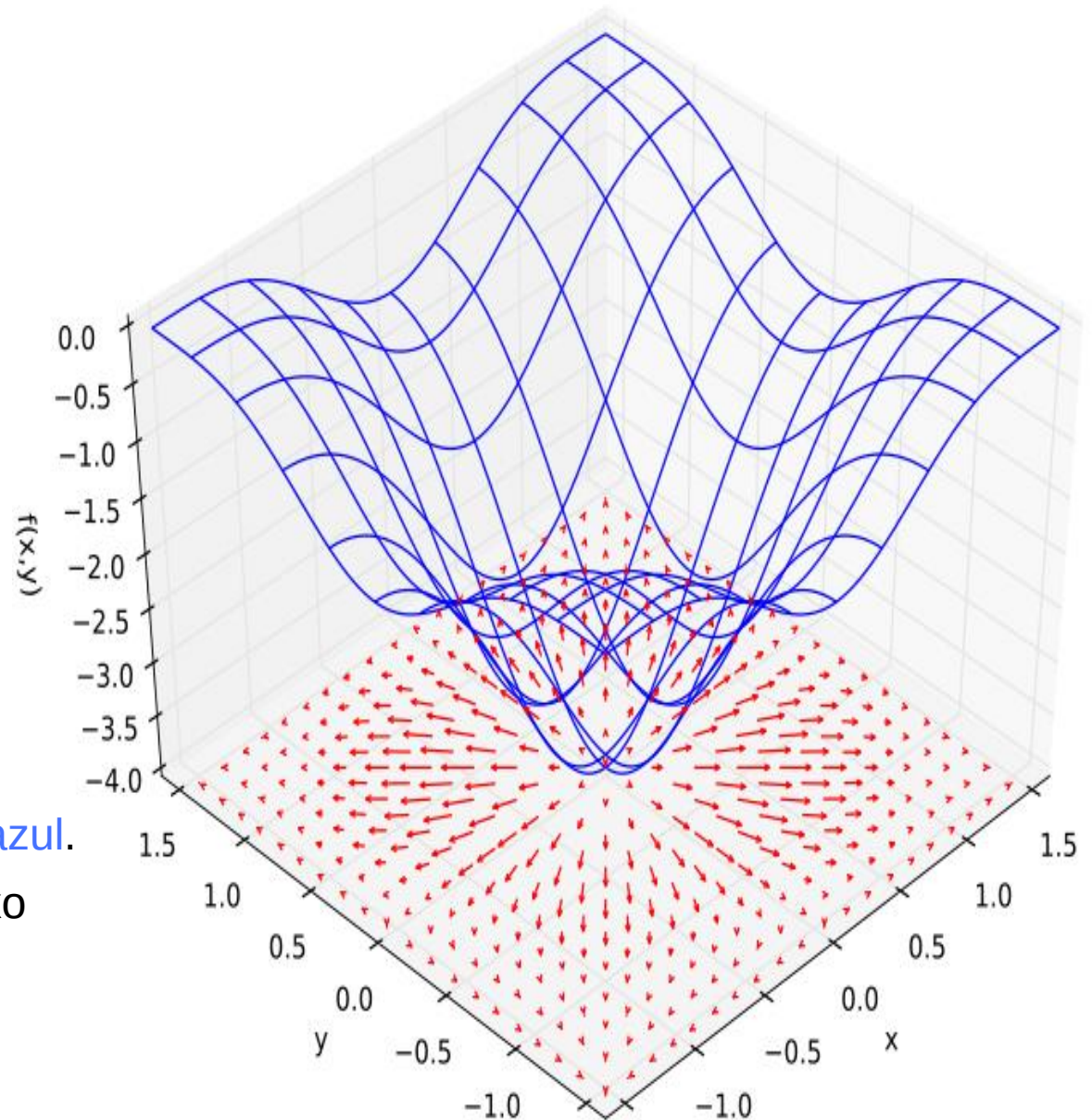
# Gradiente da Função de Perda

Quando  $\nabla_W L(W) = 0$ , então todas as derivadas parciais são nulas, ou ainda, a perda não se modifica em nenhuma direção

# Gradiente da Função de Perda

Quando  $\nabla_W L(W) = 0$ , então todas as derivadas parciais são nulas, ou ainda, a perda não se modifica em nenhuma direção

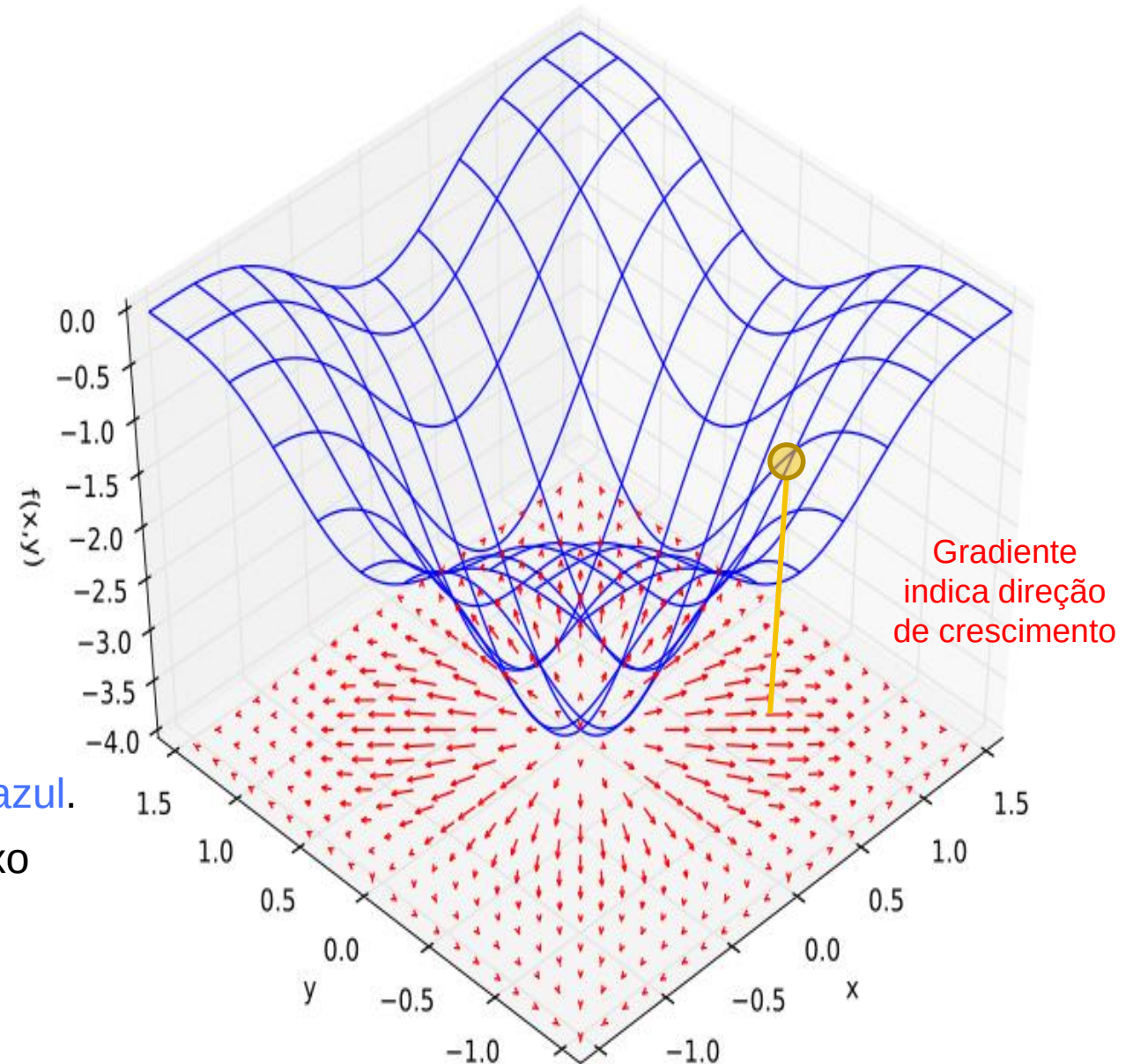
A superfície da perda aparece em azul.  
Os gradientes são mostrados abaixo  
como setas vermelhas.  
O gradiente é zero no mínimo.



# Gradiente da Função de Perda

Quando  $\nabla_W L(W) = 0$ , então todas as derivadas parciais são nulas, ou ainda, a perda não se modifica em nenhuma direção

A superfície da perda aparece em azul.  
Os gradientes são mostrados abaixo como setas vermelhas.  
O gradiente é zero no mínimo.



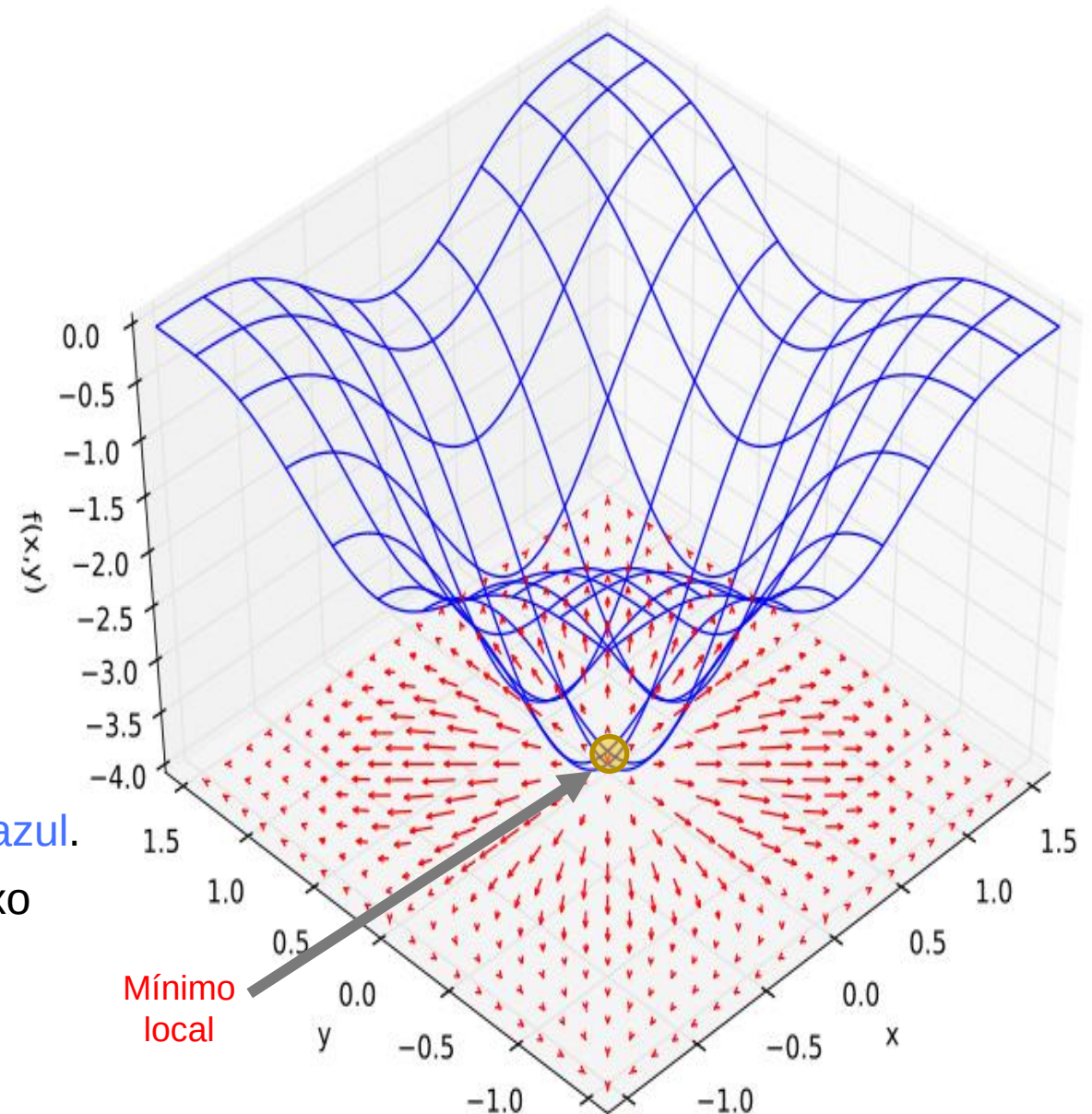


# Gradiente da Função de Perda

Quando  $\nabla_W L(W) = 0$ , então todas as derivadas parciais são nulas, ou ainda, a perda não se modifica em nenhuma direção

Portanto, obteve-se um ótimo local

A superfície da perda aparece em azul.  
Os gradientes são mostrados abaixo como setas vermelhas.  
O gradiente é zero no mínimo.

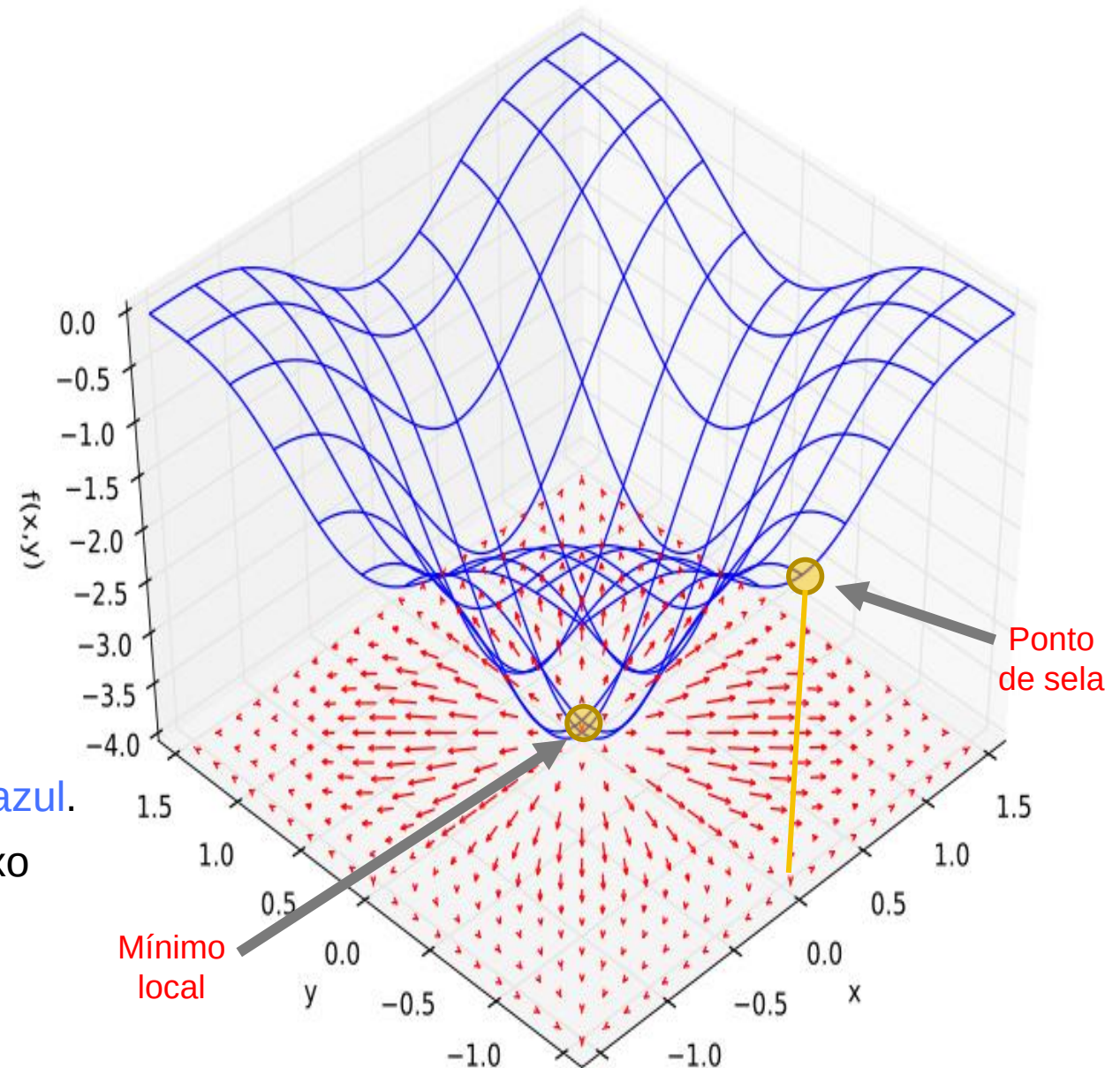


# Gradiente da Função de Perda

Quando  $\nabla_W L(W) = 0$ , então todas as derivadas parciais são nulas, ou ainda, a perda não se modifica em nenhuma direção

Portanto, obteve-se um ótimo local (ou, pelo menos, um ponto de sela)

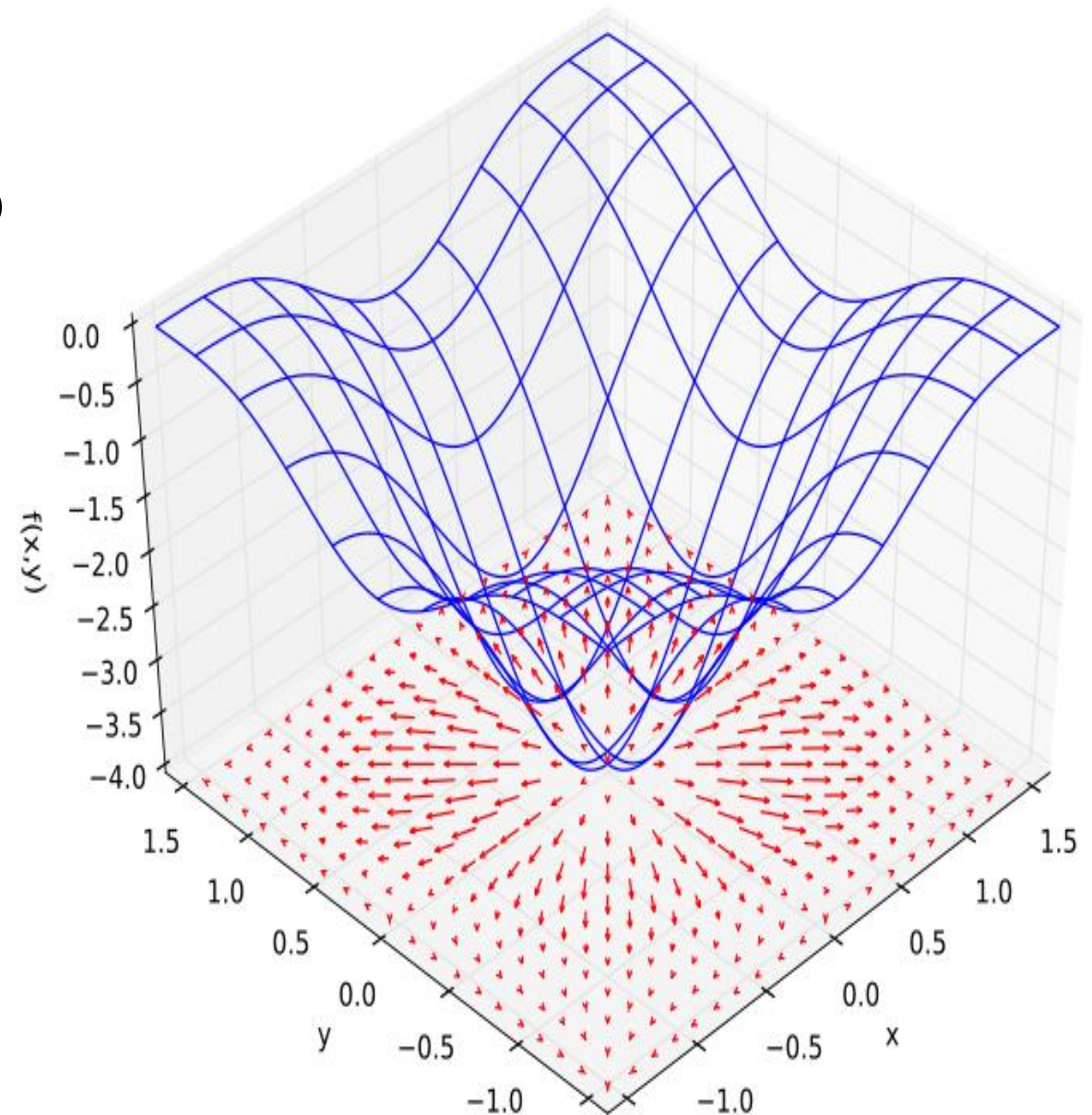
A superfície da perda aparece em azul.  
Os gradientes são mostrados abaixo como setas vermelhas.  
O gradiente é zero no mínimo.





# Método do Gradiente (ou Descida Mais Íngreme)

Para se alcançar um ponto de mínimo deve-se seguir a direção contrária ao gradiente,

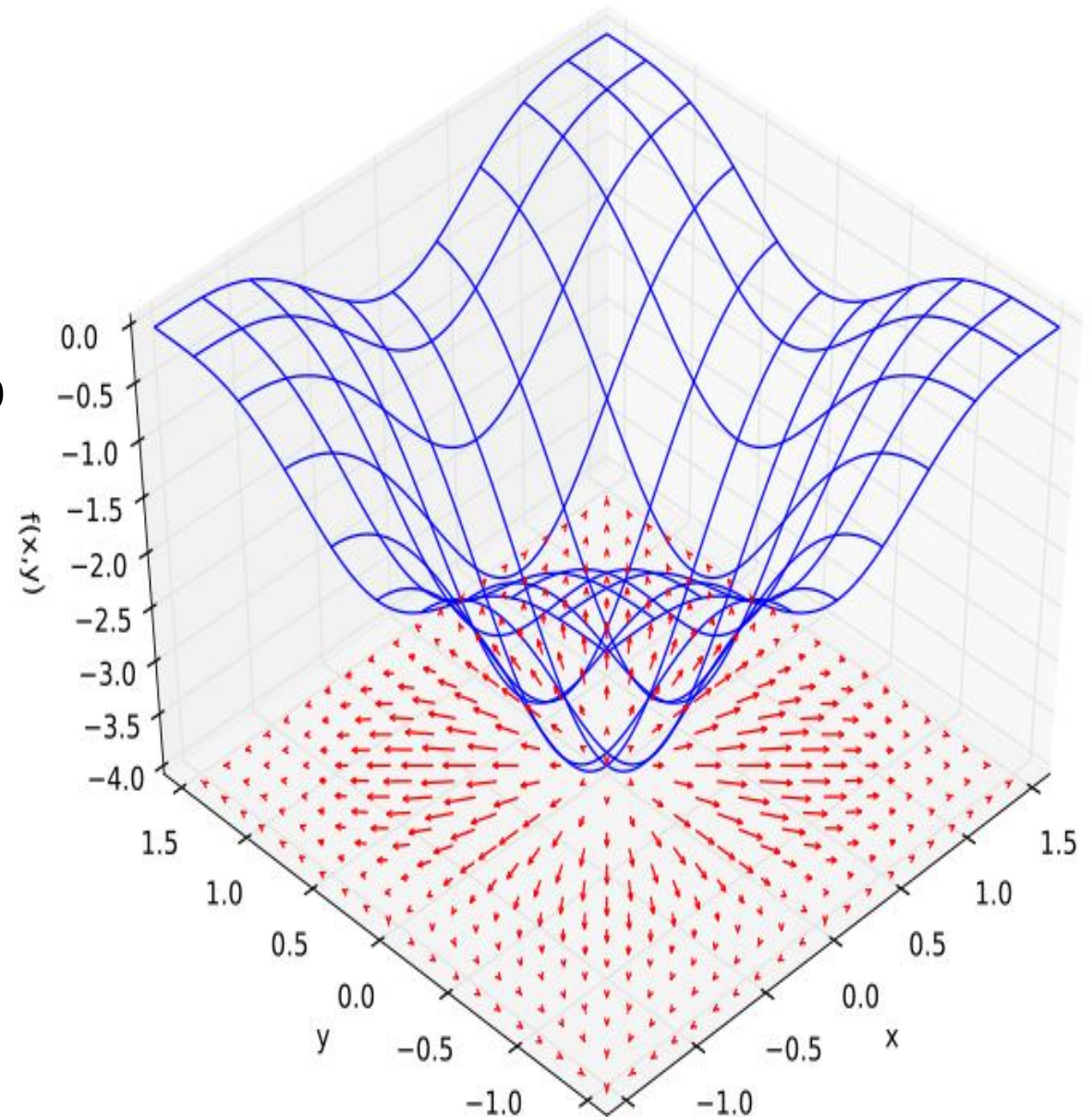


# Método do Gradiente (ou Descida Mais Íngreme)

Para se alcançar um ponto de mínimo  
deve-se seguir a direção contrária ao  
gradiente,

isto é, deve-se dar passos na direção

$$-\nabla_W L(W)$$





# Método do Gradiente (ou Descida Mais Íngreme)

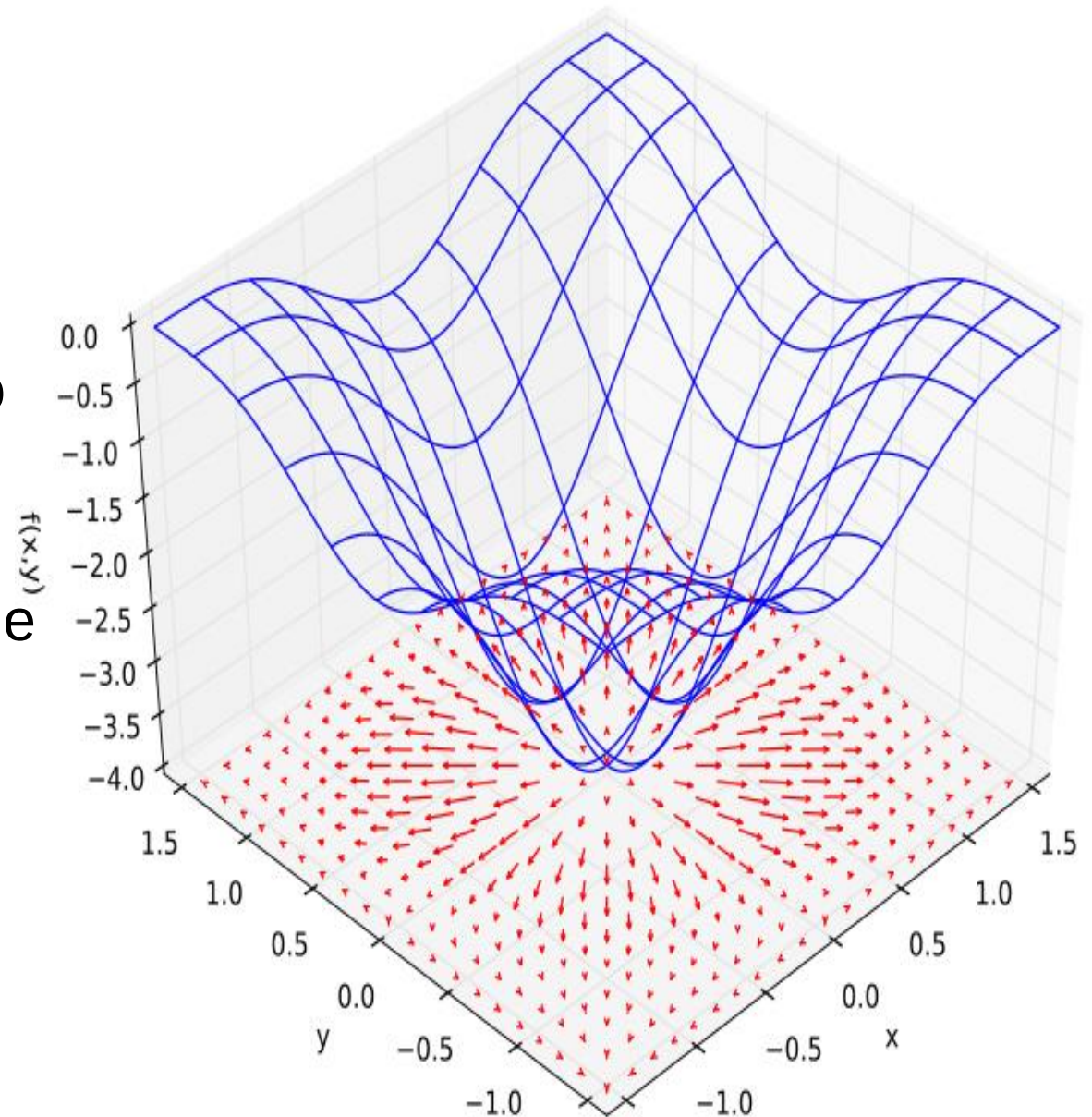
Para se alcançar um ponto de mínimo deve-se seguir a direção contrária ao gradiente,

isto é, deve-se dar passos na direção

$$-\nabla_W L(W)$$

Mais precisamente, seja  $W^t$  a matriz de pesos no passo  $t$  então

$$W^{t+1} = W^t - \alpha \nabla_W L(W)$$





# Método do Gradiente (ou Descida Mais Íngreme)

Para se alcançar um ponto de mínimo deve-se seguir a direção contrária ao gradiente,

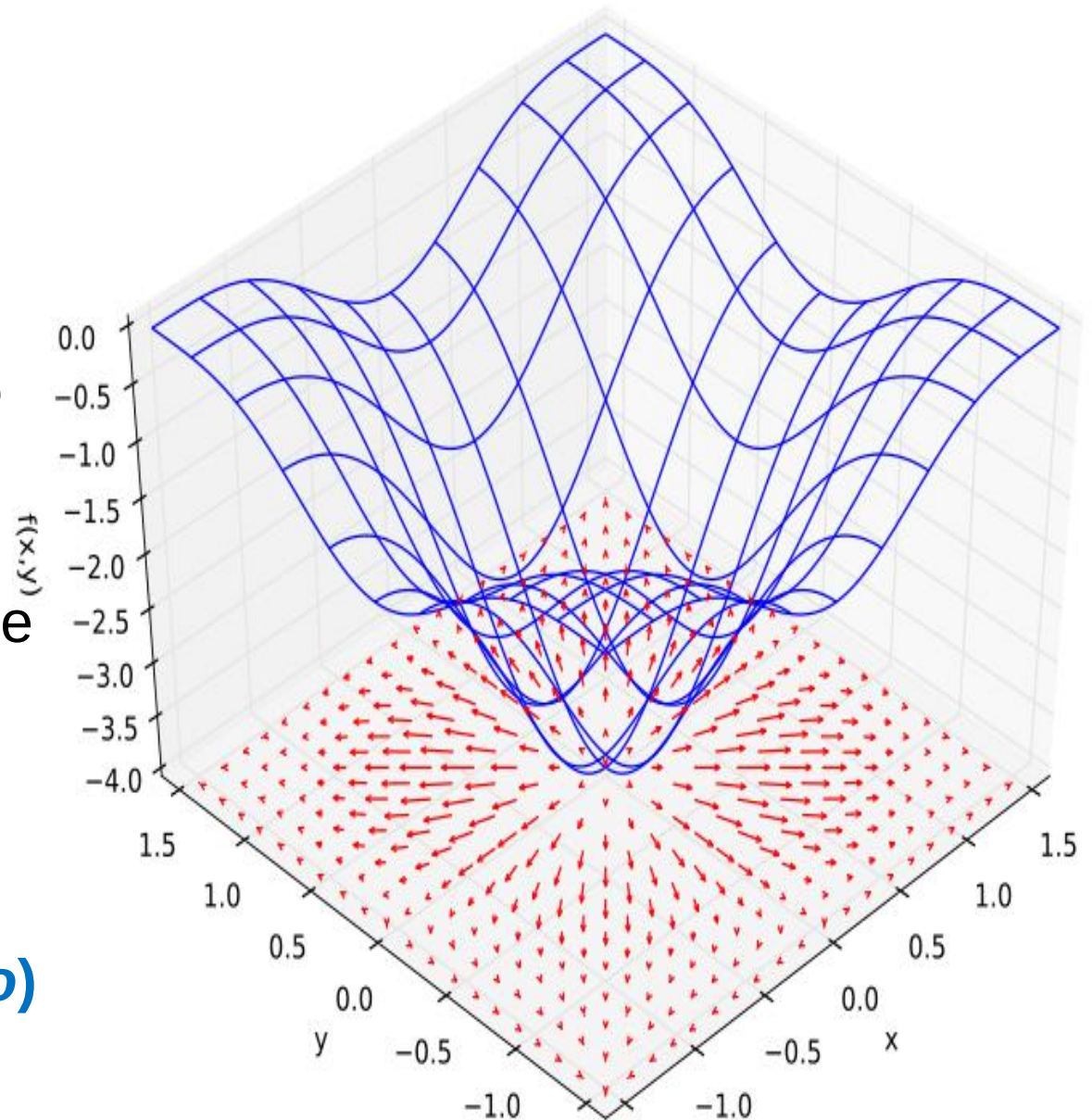
isto é, deve-se dar passos na direção

$$-\nabla_W L(W)$$

Mais precisamente, seja  $W^t$  a matriz de pesos no passo  $t$  então

$$W^{t+1} = W^t - \alpha \nabla_W L(W)$$

em que  $\alpha$  é chamado de **taxa de aprendizado (ou tamanho do passo)**



# Método do Gradiente (ou Descida Mais Íngreme)

Para se alcançar um ponto de mínimo deve-se seguir a direção contrária ao gradiente,

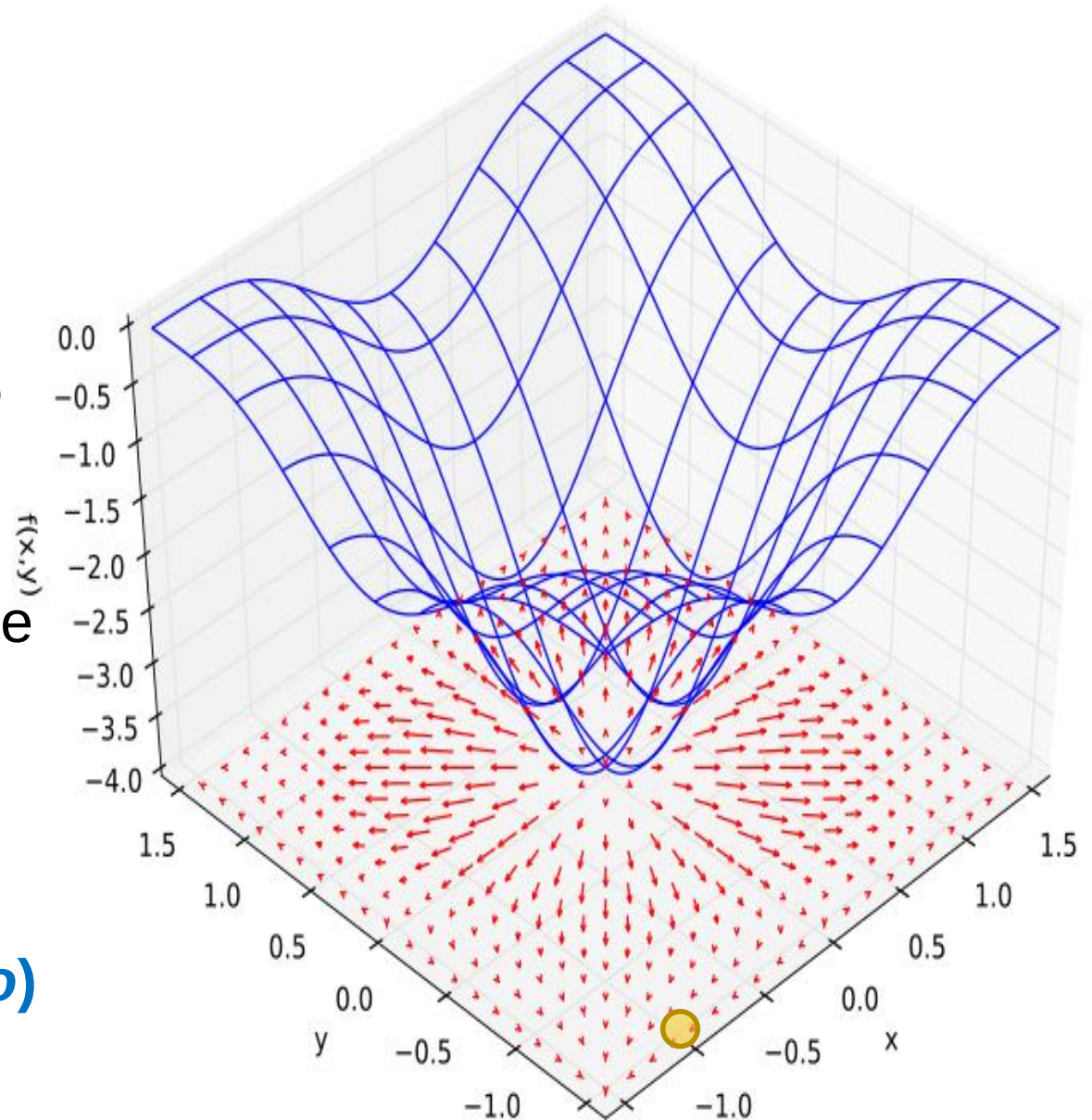
isto é, deve-se dar passos na direção

$$-\nabla_W L(W)$$

Mais precisamente, seja  $W^t$  a matriz de pesos no passo  $t$  então

$$W^{t+1} = W^t - \alpha \nabla_W L(W)$$

em que  $\alpha$  é chamado de **taxa de aprendizado (ou tamanho do passo)**



# Método do Gradiente (ou Descida Mais Íngreme)

```
# Vanilla Gradient Descent
```

```
while True:
```

```
    weights_grad = evaluate_gradient(loss_fun, data, weights)
```

```
    weights += - step_size * weights_grad # perform parameter update
```

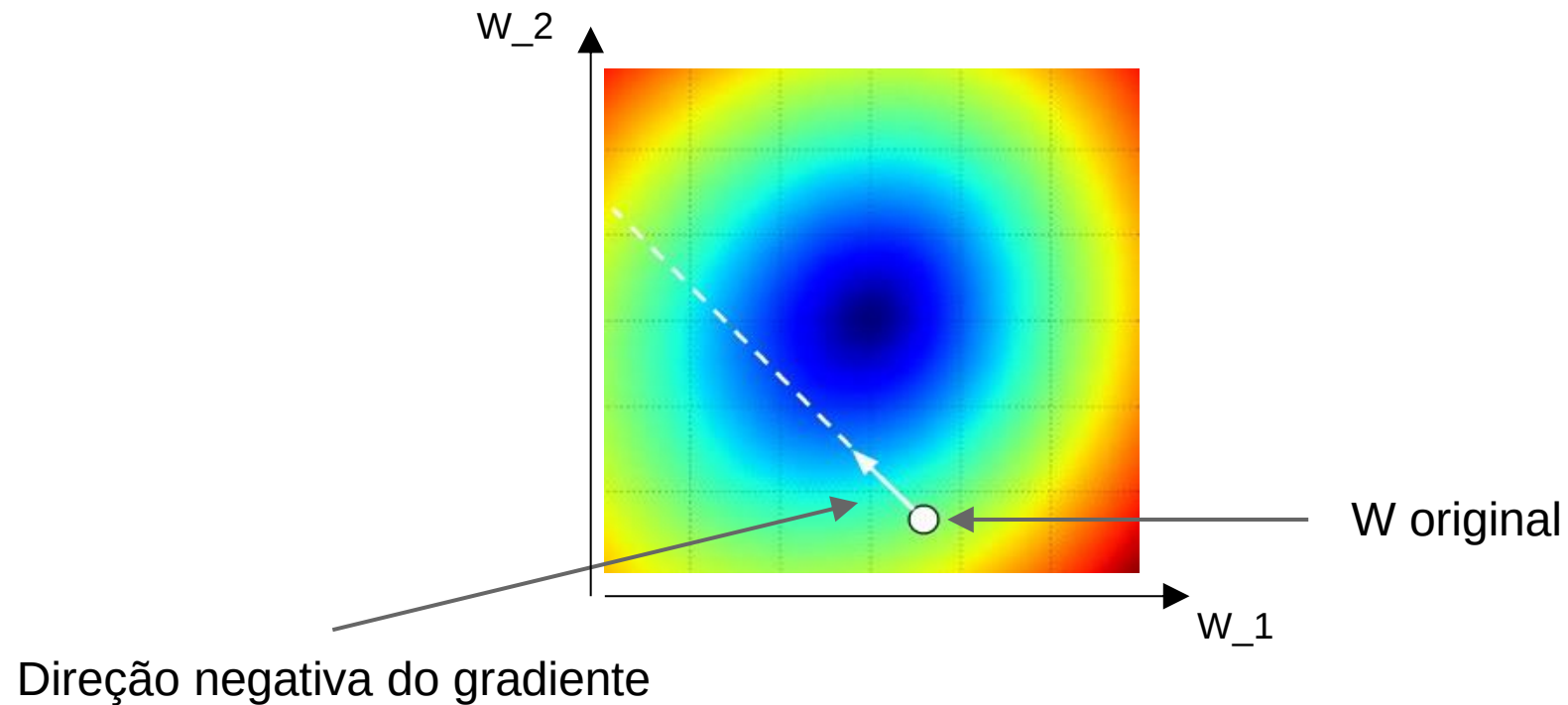
# Método do Gradiente (ou Descida Mais Íngreme)

```
# Vanilla Gradient Descent
```

```
while True:
```

```
    weights_grad = evaluate_gradient(loss_fun, data, weights)
```

```
    weights += - step_size * weights_grad # perform parameter update
```





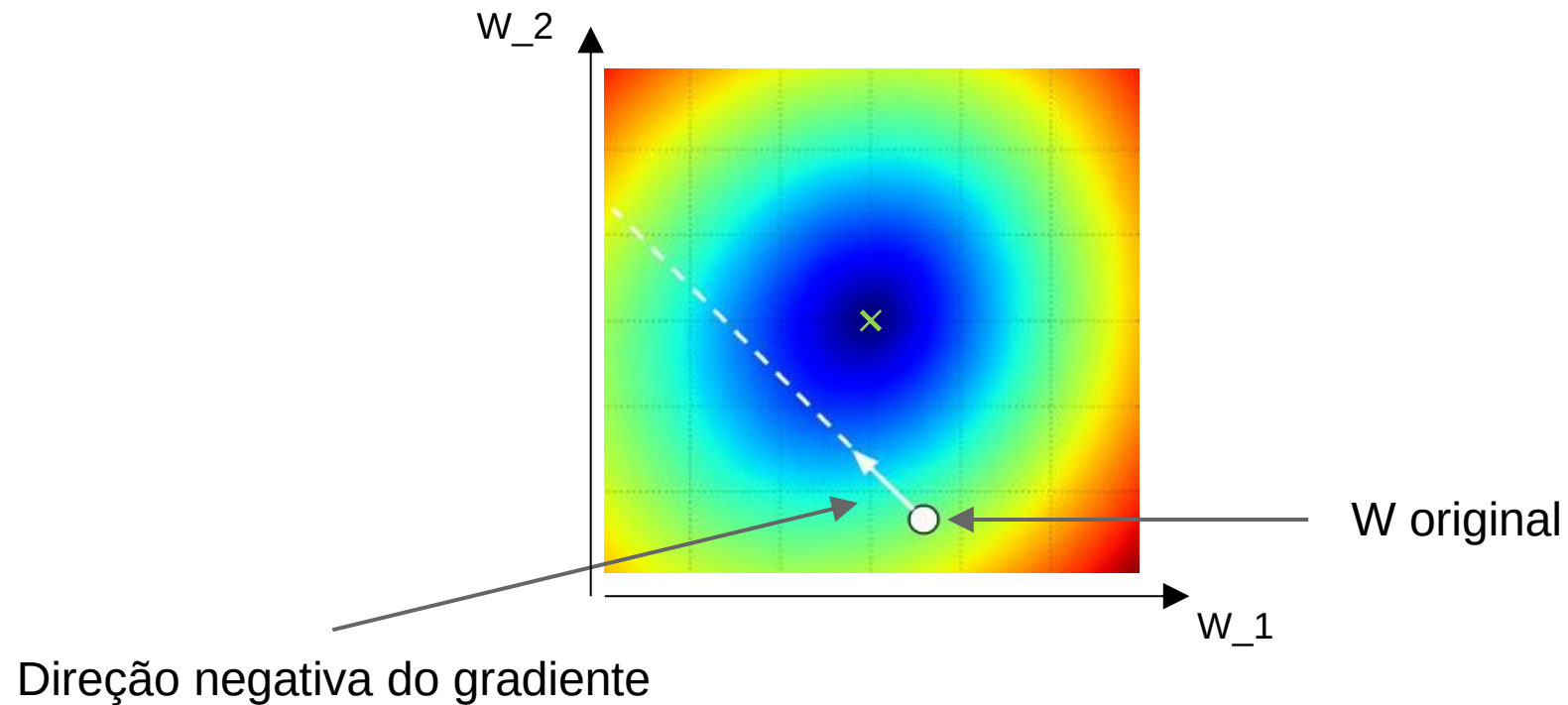
# Método do Gradiente (ou Descida Mais Íngreme)

```
# Vanilla Gradient Descent
```

```
while True:
```

```
    weights_grad = evaluate_gradient(loss_fun, data, weights)
```

```
    weights += - step_size * weights_grad # perform parameter update
```



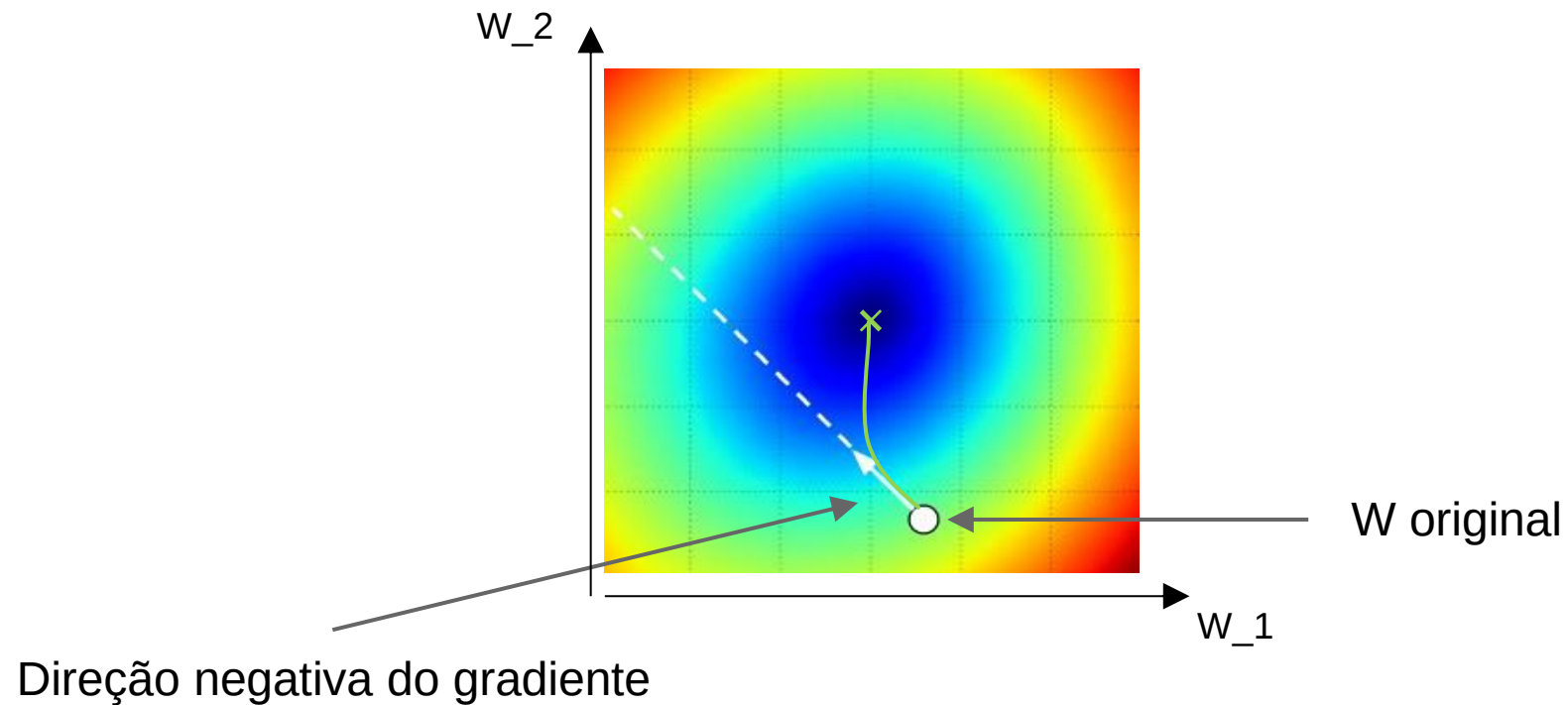
# Método do Gradiente (ou Descida Mais Íngreme)

```
# Vanilla Gradient Descent
```

```
while True:
```

```
    weights_grad = evaluate_gradient(loss_fun, data, weights)
```

```
    weights += - step_size * weights_grad # perform parameter update
```





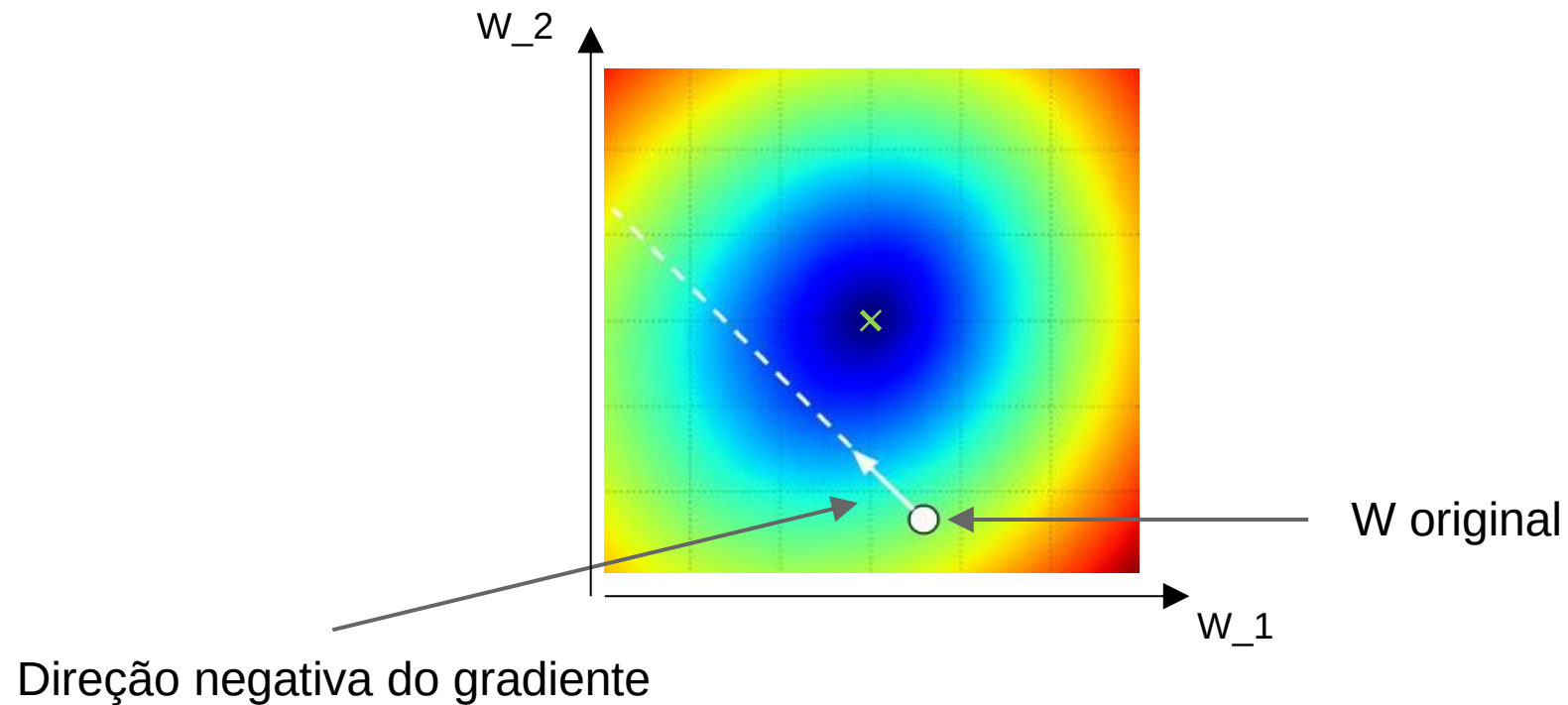
# Método do Gradiente (ou Descida Mais Íngreme)

```
# Vanilla Gradient Descent
```

```
while True:
```

```
    weights_grad = evaluate_gradient(loss_fun, data, weights)
```

```
    weights += - step_size * weights_grad # perform parameter update
```



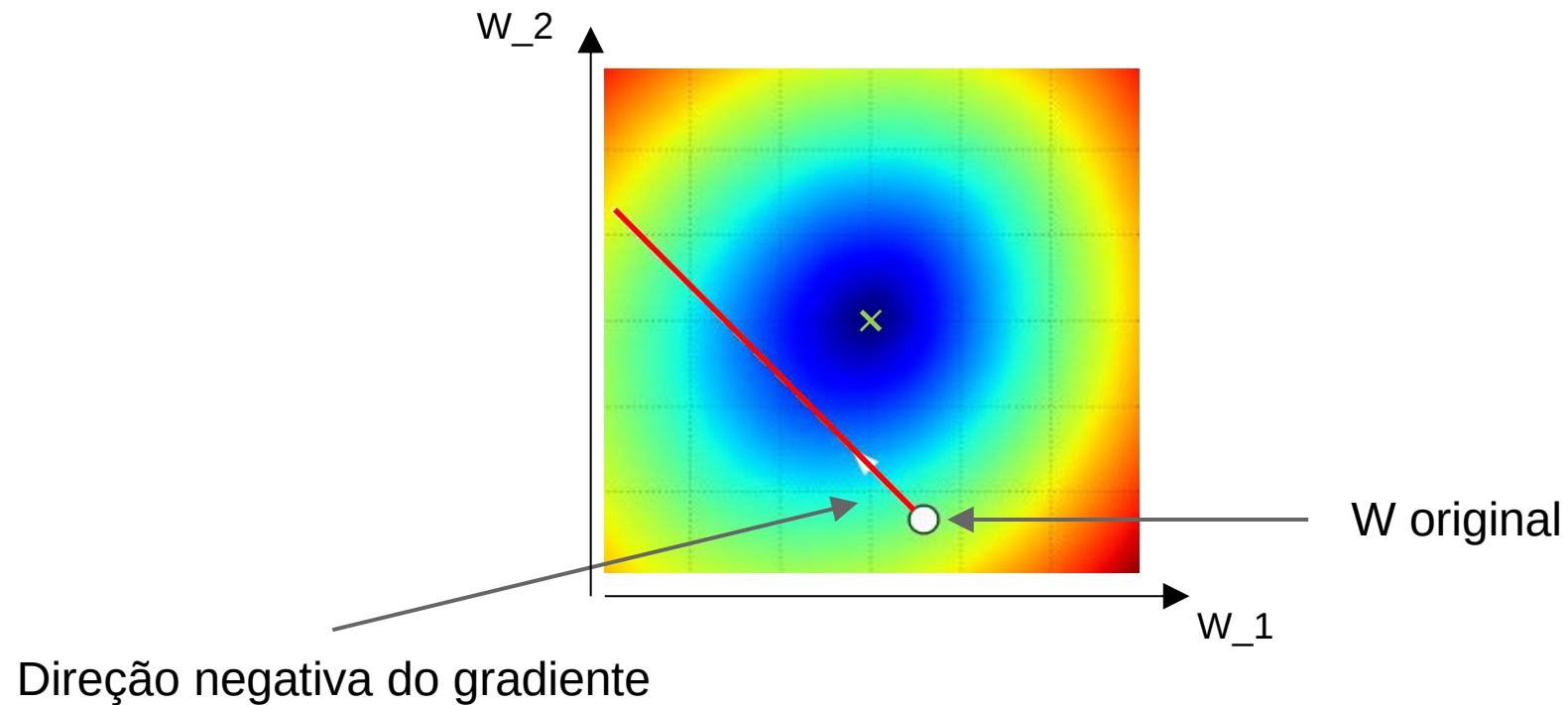
# Método do Gradiente (ou Descida Mais Íngreme)

```
# Vanilla Gradient Descent
```

```
while True:
```

```
    weights_grad = evaluate_gradient(loss_fun, data, weights)
```

```
    weights += - step_size * weights_grad # perform parameter update
```



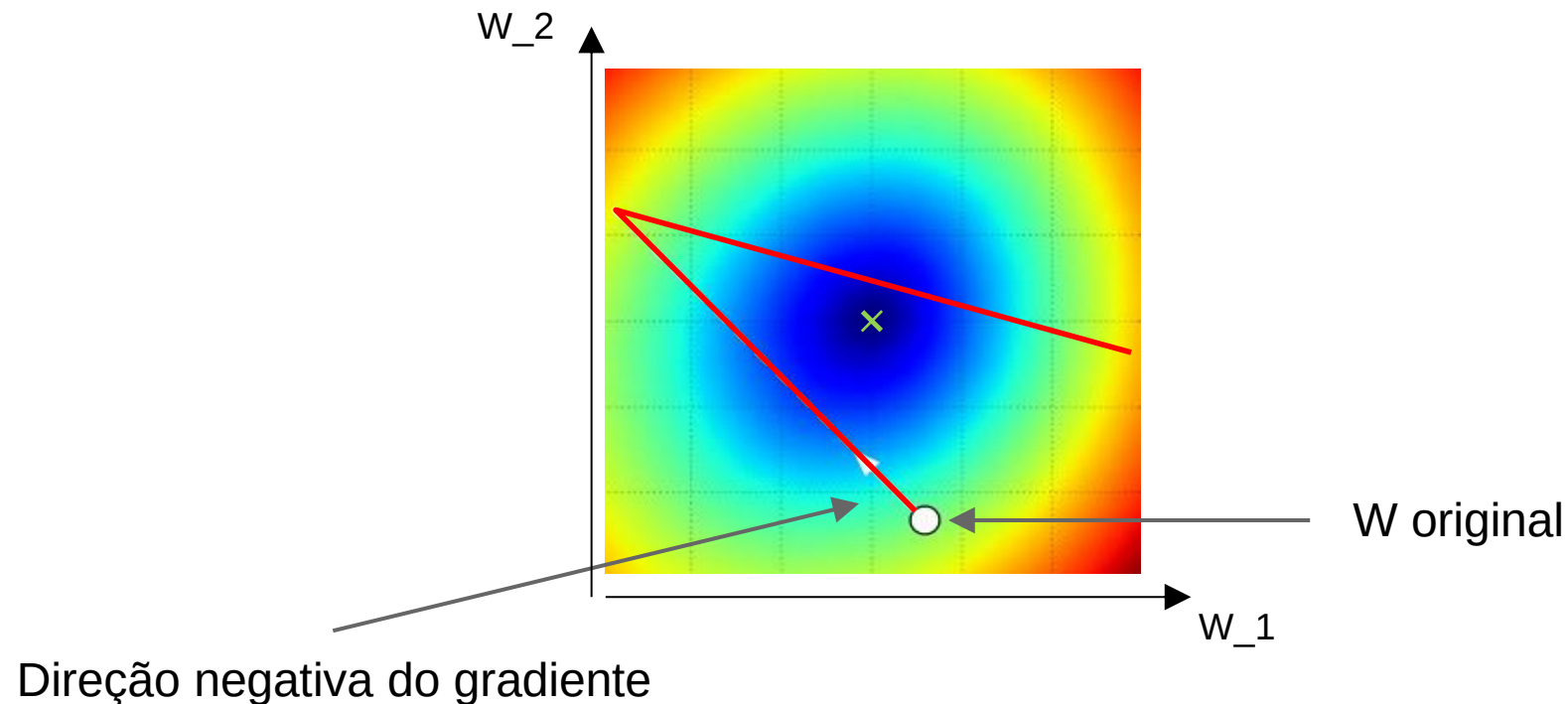
# Método do Gradiente (ou Descida Mais Íngreme)

```
# Vanilla Gradient Descent
```

```
while True:
```

```
    weights_grad = evaluate_gradient(loss_fun, data, weights)
```

```
    weights += - step_size * weights_grad # perform parameter update
```



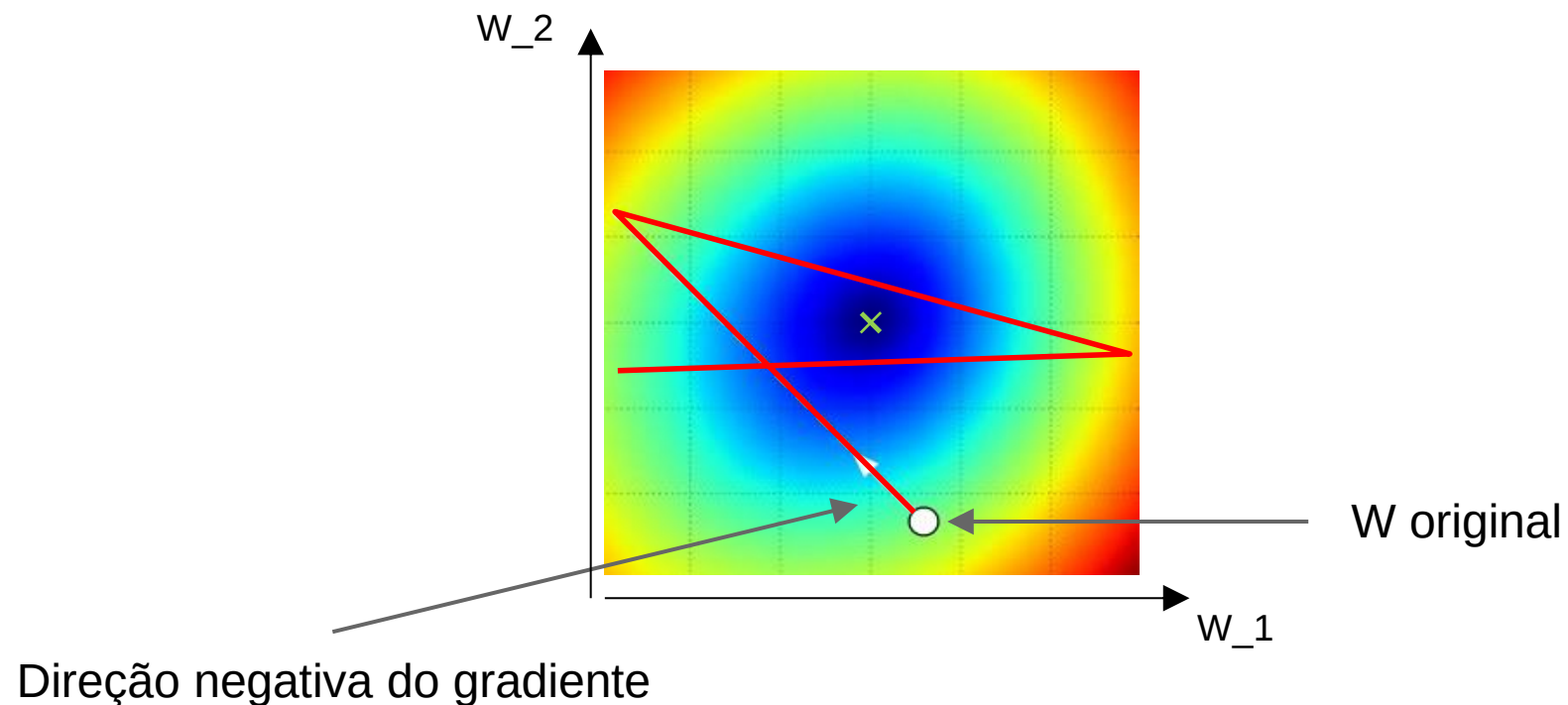
# Método do Gradiente (ou Descida Mais Íngreme)

```
# Vanilla Gradient Descent
```

```
while True:
```

```
    weights_grad = evaluate_gradient(loss_fun, data, weights)
```

```
    weights += - step_size * weights_grad # perform parameter update
```



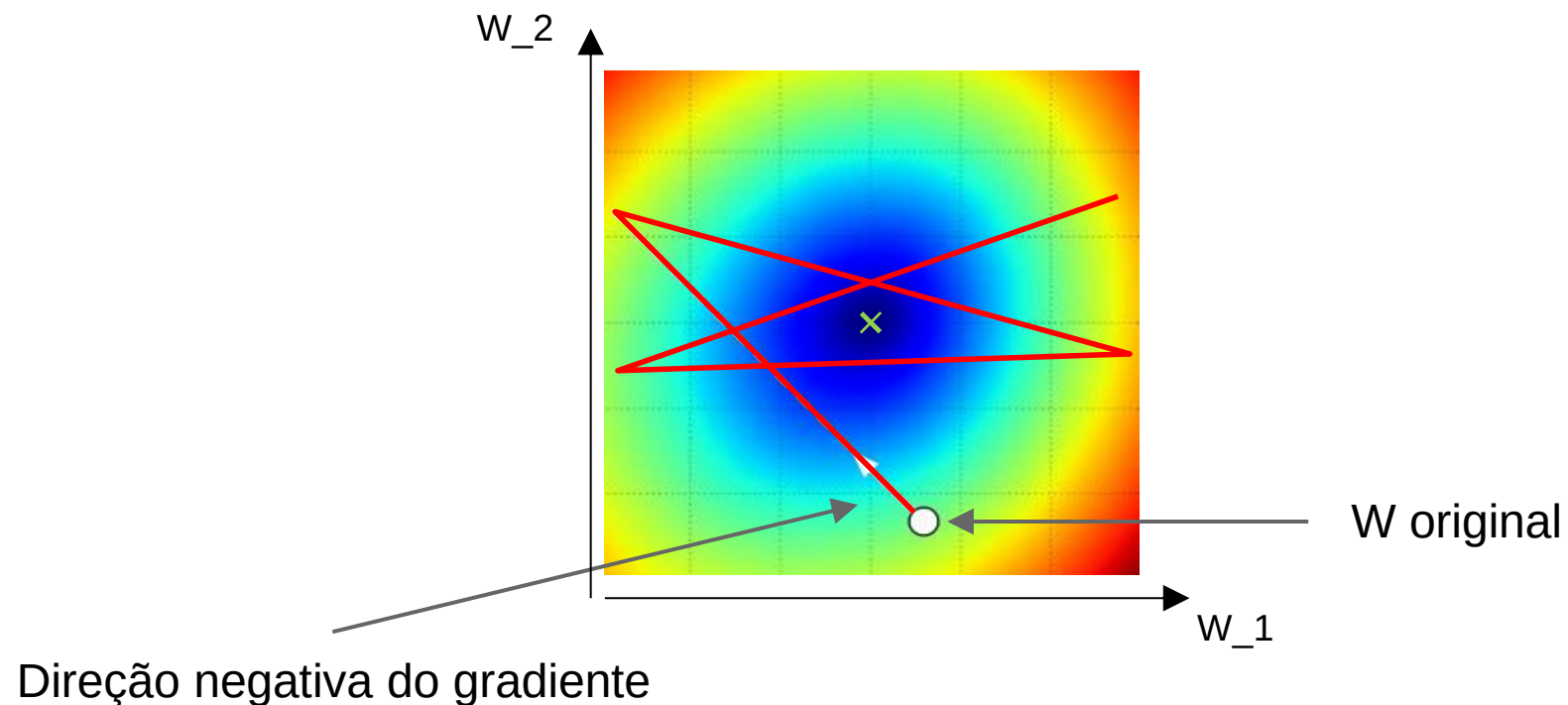
# Método do Gradiente (ou Descida Mais Íngreme)

```
# Vanilla Gradient Descent
```

```
while True:
```

```
    weights_grad = evaluate_gradient(loss_fun, data, weights)
```

```
    weights += - step_size * weights_grad # perform parameter update
```



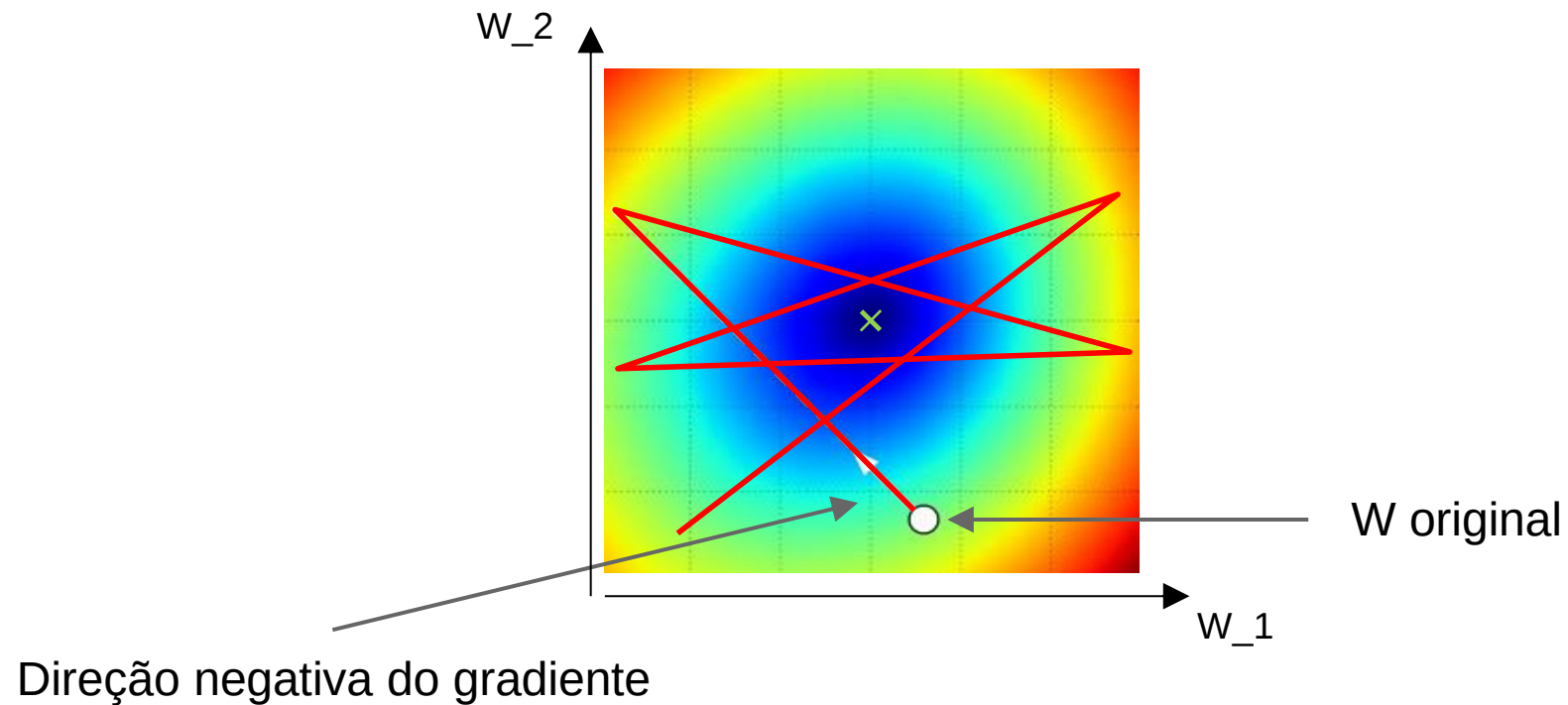
# Método do Gradiente (ou Descida Mais Íngreme)

```
# Vanilla Gradient Descent
```

```
while True:
```

```
    weights_grad = evaluate_gradient(loss_fun, data, weights)
```

```
    weights += - step_size * weights_grad # perform parameter update
```





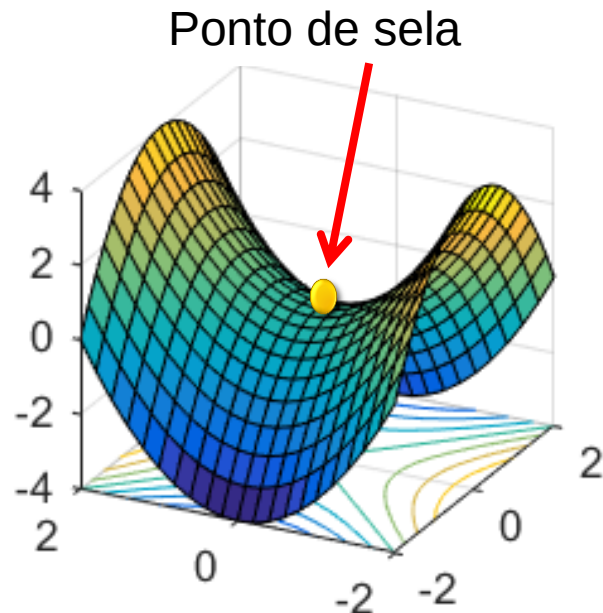
# Ressalva: Pontos de Sela

Seguir a direção contrária ao gradiente deve levar a uma perda mínima

# Ressalva: Pontos de Sela

Seguir a direção contrária ao gradiente deve levar a uma perda mínima

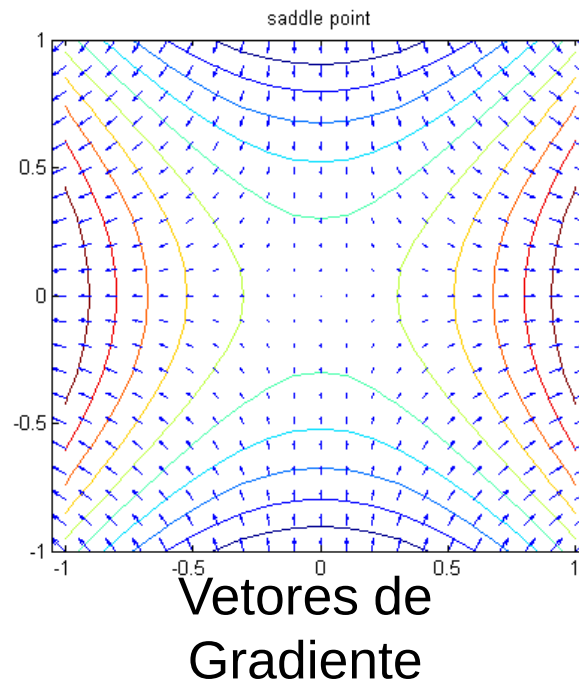
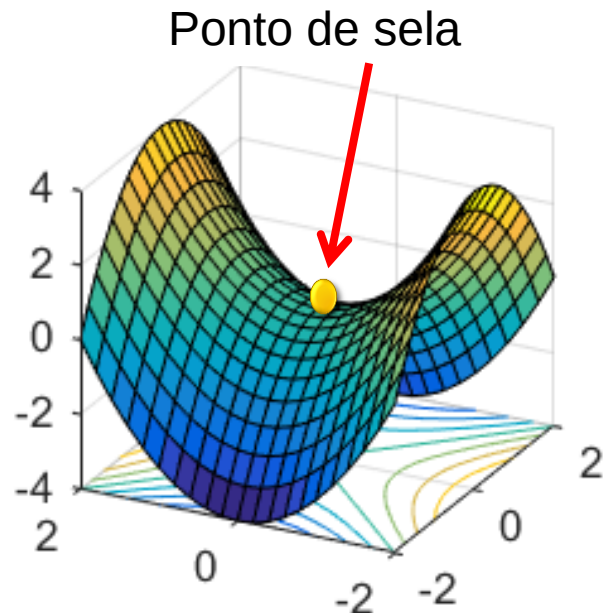
Mas pode demorar muito tempo. Uma razão é a presença de pontos de sela, em que o gradiente também desaparece



# Ressalva: Pontos de Sela

Seguir a direção contrária ao gradiente deve levar a uma perda mínima

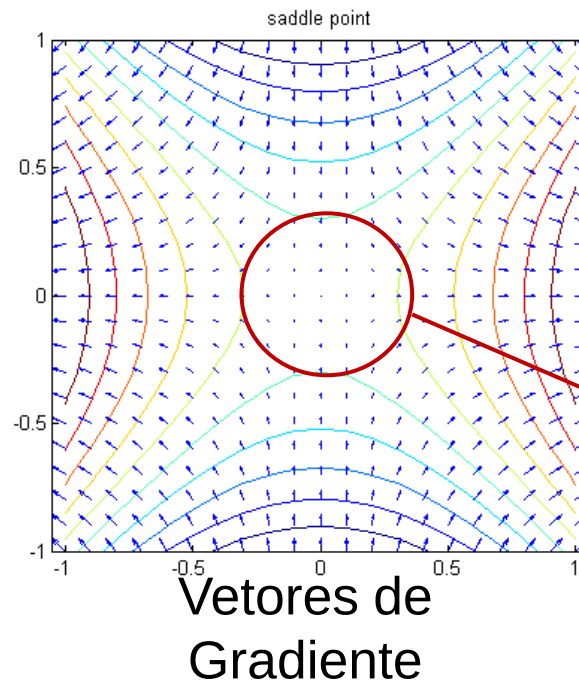
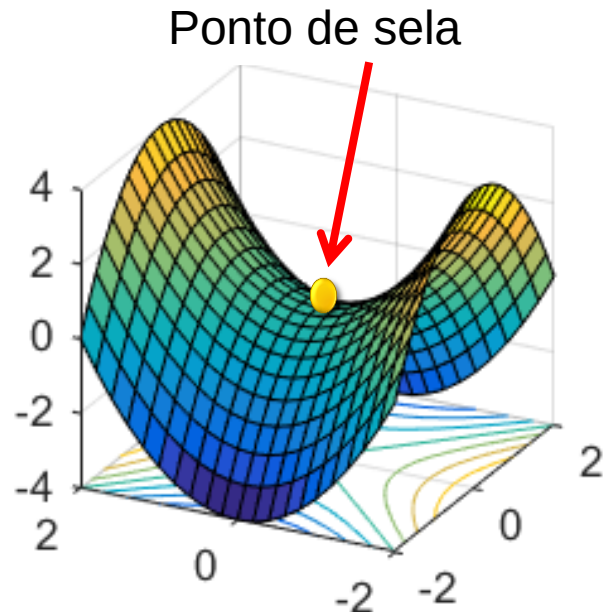
Mas pode demorar muito tempo. Uma razão é a presença de pontos de sela, em que o gradiente também desaparece



# Ressalva: Pontos de Sela

Seguir a direção contrária ao gradiente deve levar a uma perda mínima

Mas pode demorar muito tempo. Uma razão é a presença de pontos de sela, em que o gradiente também desaparece



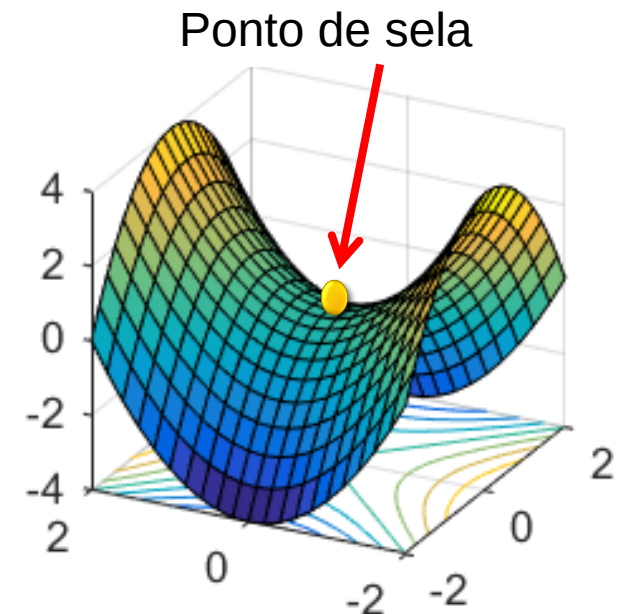
Gradientes são muito pequenos aqui – ocasionando progresso muito lento

# Ressalva: Pontos de Sela

Existem várias evidências de que a maioria dos extremos da função de perda (ou zeros do gradiente da função de perda  $\nabla_W L$ ) para redes neurais profundas são de fato pontos de sela

Veja por exemplo

Dauphin et al. “Identifying and attacking the saddle point problem in high-dimensional non-convex optimization”  
arXiv 1406.2572, 2014.



# Ressalva : Eficiência

A perda total  $L$  é dada pela soma das perdas para todos os itens de dados

$$L = \sum_{i=1}^N L(x_i, y_i, W)$$



# Ressalva : Eficiência

A perda total  $L$  é dada pela soma das perdas para todos os itens de dados

$$L = \sum_{i=1}^N L(x_i, y_i, W)$$

assim o gradiente da perda total é dado por

$$\frac{dL}{dW} = \sum_{i=1}^N \frac{dL}{dW}(x_i, y_i, W)$$

# Ressalva : Eficiência

A perda total  $L$  é dada pela soma das perdas para todos os itens de dados

$$L = \sum_{i=1}^N L(x_i, y_i, W)$$

assim o gradiente da perda total é dado por

$$\frac{dL}{dW} = \sum_{i=1}^N \frac{dL}{dW}(x_i, y_i, W)$$

Portanto, calcular o gradiente em relação a  $W$  exige uma **passagem completa pelo conjunto de dados**

# Ressalva : Eficiência

A perda total  $L$  é dada pela soma das perdas para todos os itens de dados

$$L = \sum_{i=1}^N L(x_i, y_i, W)$$

assim o gradiente da perda total é dado por

$$\frac{dL}{dW} = \sum_{i=1}^N \frac{dL}{dW}(x_i, y_i, W)$$

Portanto, calcular o gradiente em relação a  $W$  exige uma **passagem completa pelo conjunto de dados**

A obtenção do valor mínimo da função de perda pode exigir milhões de passos de gradiente e, portanto, se torna muito caro

# Processamento em Lotes (*Minibatches*)

Em vez de se calcular um gradiente sobre todo o conjunto de dados, pode-se calculá-lo usando um subconjunto de tamanho fixo de amostras de dados chamado *minibatch* (seu tamanho é tipicamente 32, 64, 128, 256,...)

# Processamento em Lotes (*Minibatches*)

Em vez de se calcular um gradiente sobre todo o conjunto de dados, pode-se calculá-lo usando um subconjunto de tamanho fixo de amostras de dados chamado **minibatch** (seu tamanho é tipicamente 32, 64, 128, 256,...)

O **minibatch** é idealmente uma amostra aleatória de tamanho  $m$  do conjunto de dados (na prática, podem ser apenas  $m$  amostras consecutivas)

# Processamento em Lotes (*Minibatches*)

Em vez de se calcular um gradiente sobre todo o conjunto de dados, pode-se calculá-lo usando um subconjunto de tamanho fixo de amostras de dados chamado **minibatch** (seu tamanho é tipicamente 32, 64, 128, 256,...)

O **minibatch** é idealmente uma amostra aleatória de tamanho  $m$  do conjunto de dados (na prática, podem ser apenas  $m$  amostras consecutivas)

Então se calcula o gradiente da seguinte forma: ( $N$  o tamanho do conjunto de dados,  $m$  o tamanho do **minibatch**)

$$g^{(t)} = \frac{1}{m} \sum_{j = i_1, \dots, i_m \in \{1, \dots, N\}} \nabla_W L(x_j, y_j, W)$$



# Gradiente Descendente Estocástico (SGD)

Considerando  $W^{(t)}$  como a matriz de pesos na iteração  $t$ , temos que:

$$W^{(t+1)} = W^{(t)} - \alpha g^{(t)}$$

# Gradiente Descendente Estocástico (SGD)

Considerando  $W^{(t)}$  como a matriz de pesos na iteração  $t$ , temos que:

$$W^{(t+1)} = W^{(t)} - \alpha g^{(t)}$$

Esse método é chamado de **gradiente descendente estocástico** (SGD). O SGD e suas variantes são usadas quase universalmente em treinamento de redes profundas

# Gradiente Descendente Estocástico (SGD)

Considerando  $W^{(t)}$  como a matriz de pesos na iteração  $t$ , temos que:

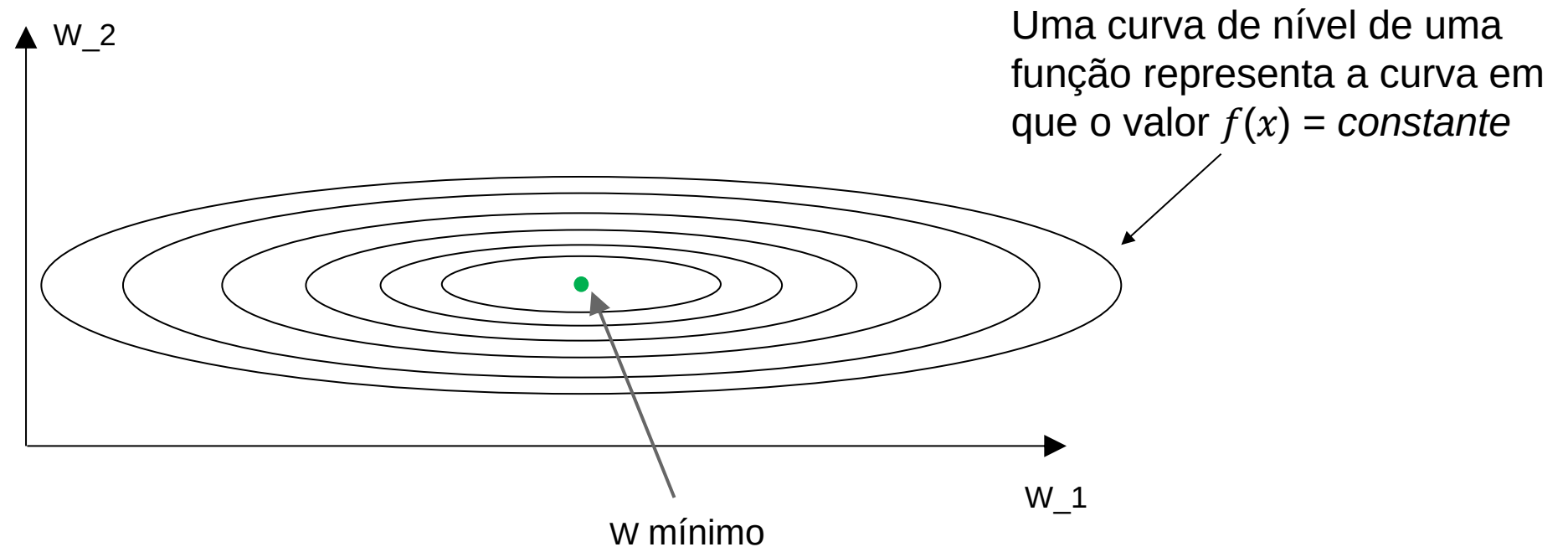
$$W^{(t+1)} = W^{(t)} - \alpha g^{(t)}$$

Esse método é chamado de **gradiente descendente estocástico** (SGD). O SGD e suas variantes são usadas quase universalmente em treinamento de redes profundas

O SGD usa  $g^{(t)}$ , isto é o gradiente de um **minibatch**, no lugar do gradiente sobre o conjunto de dados completo e é dito “estocástico” pois o gradiente de um **minibatch** é calculado sobre uma amostra do conjunto de dados

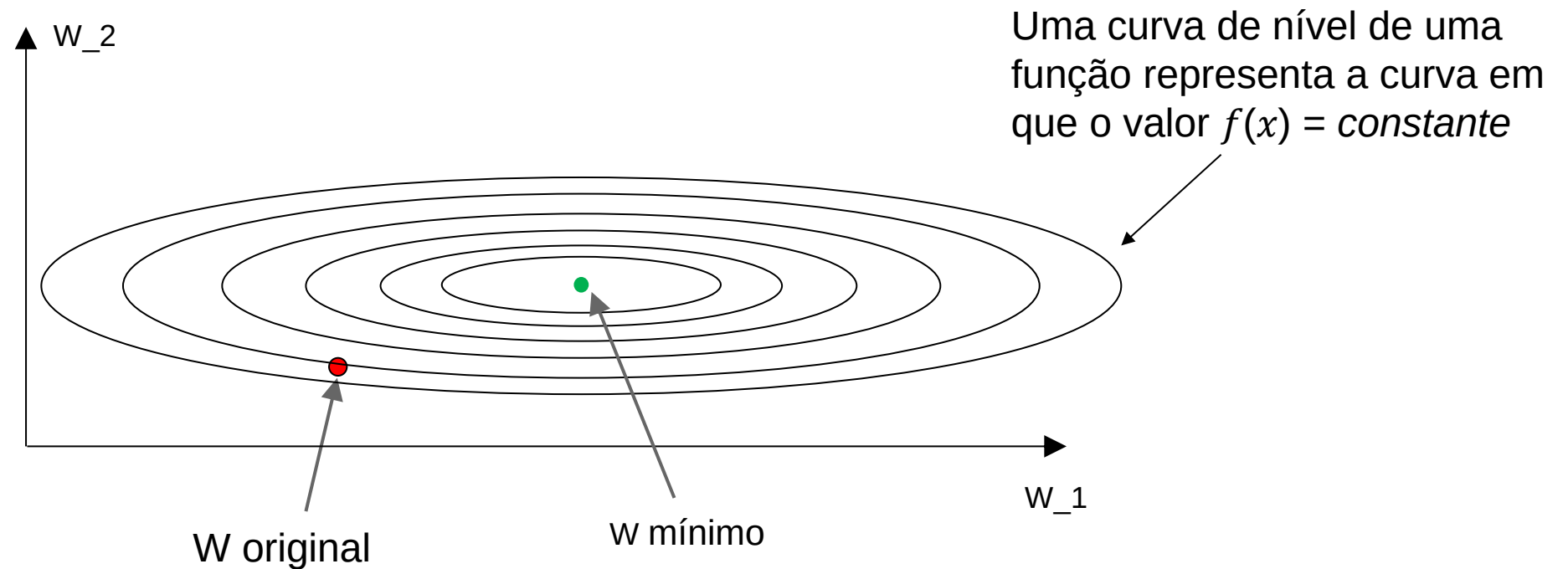
# Gradiente Descendente Estocástico (SGD)

O gradiente da função em um ponto é sempre perpendicular a curva de nível naquele ponto



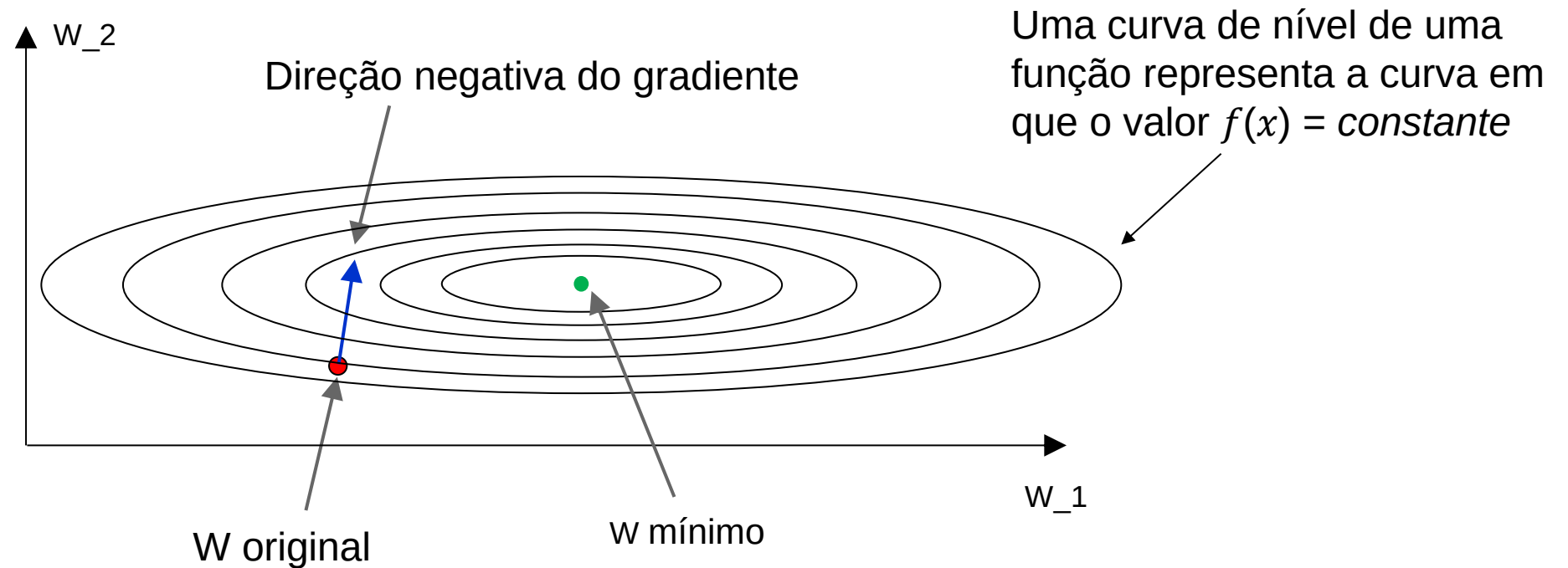
# Gradiente Descendente Estocástico (SGD)

O gradiente da função em um ponto é sempre perpendicular a curva de nível naquele ponto

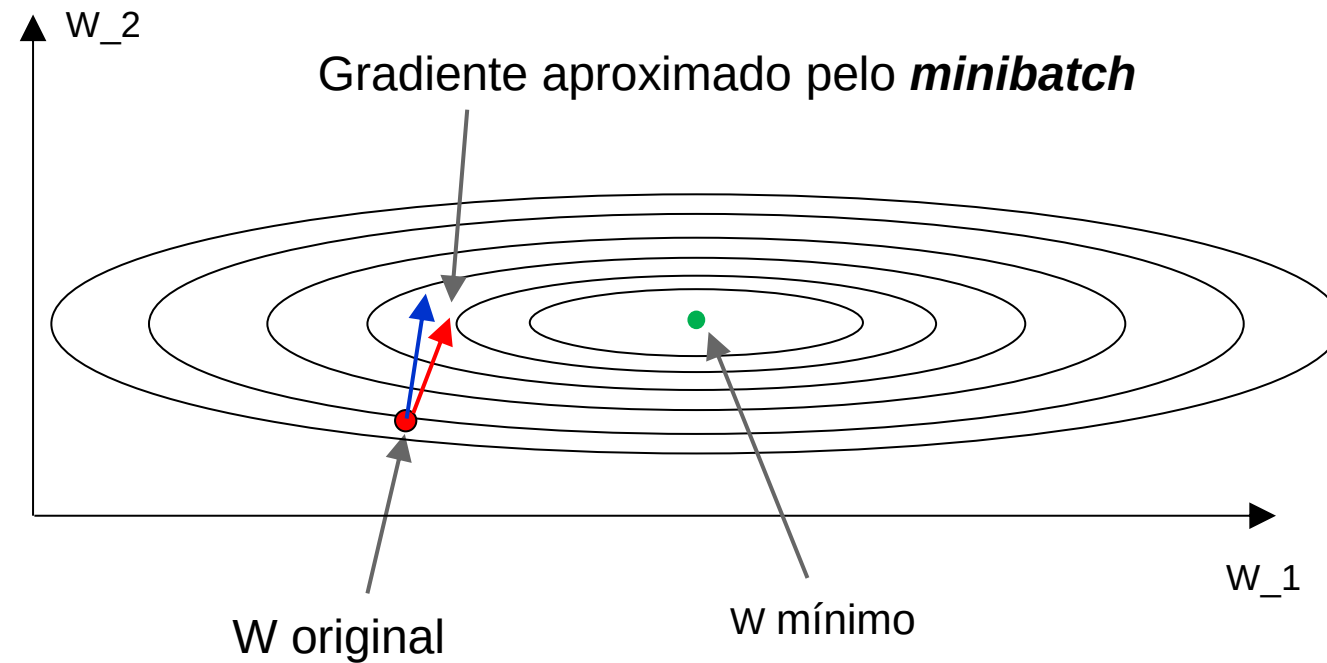


# Gradiente Descendente Estocástico (SGD)

O gradiente da função em um ponto é sempre perpendicular a curva de nível naquele ponto



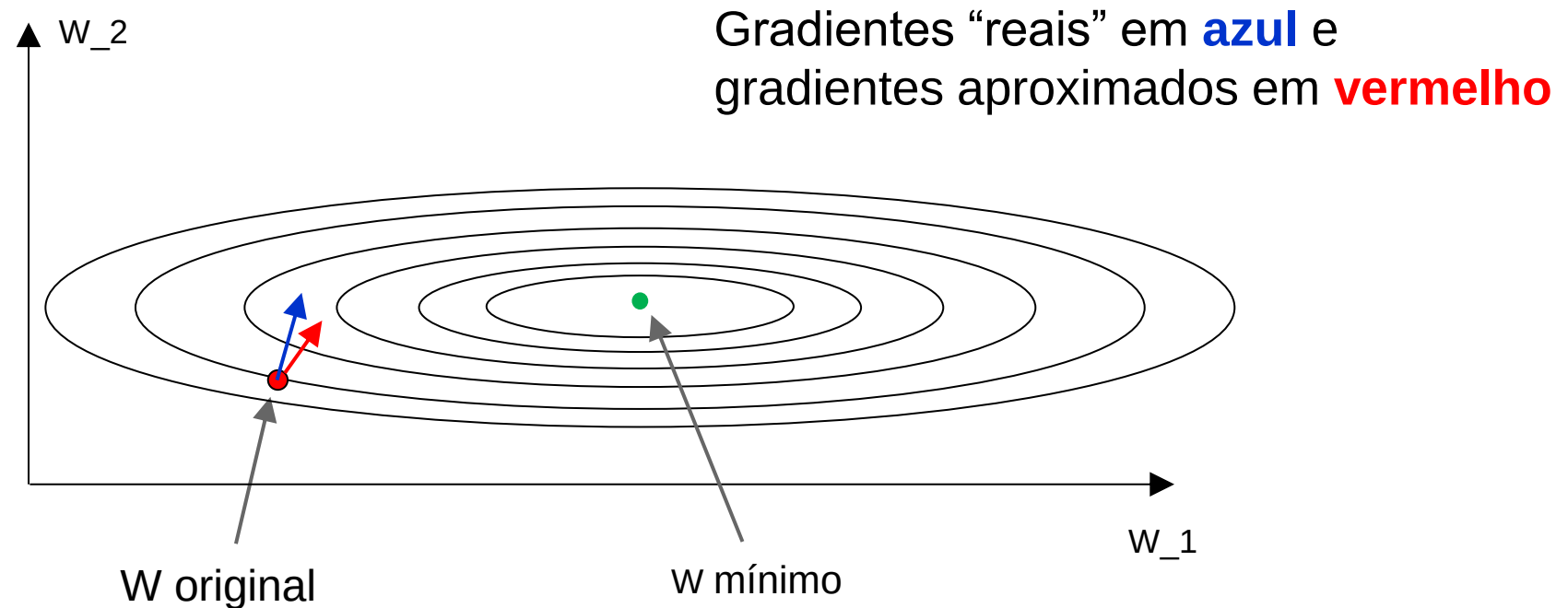
# Gradiente Descendente Estocástico (SGD)





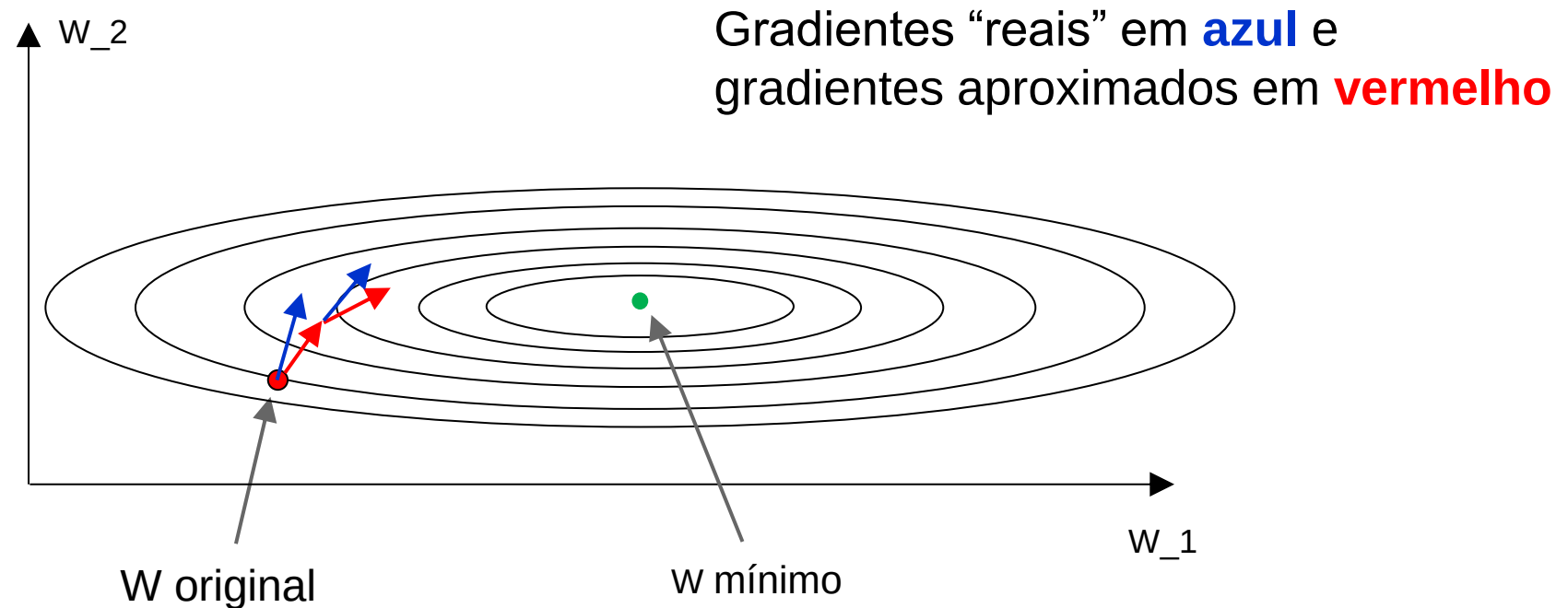
# Gradiente Descendente Estocástico (SGD)

Gradientes apresentam “ruídos” mas ainda são bons para progredir na média em direção ao mínimo



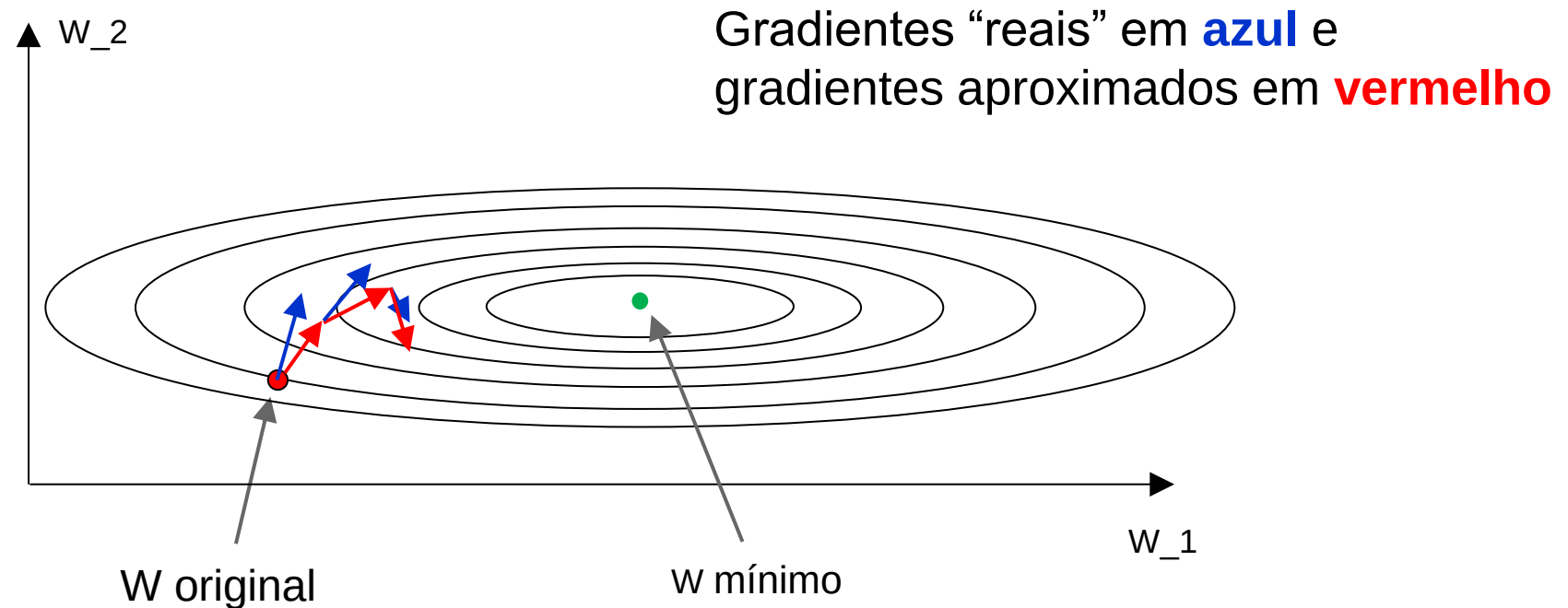
# Gradiente Descendente Estocástico (SGD)

Gradientes apresentam “ruídos” mas ainda são bons para progredir na média em direção ao mínimo



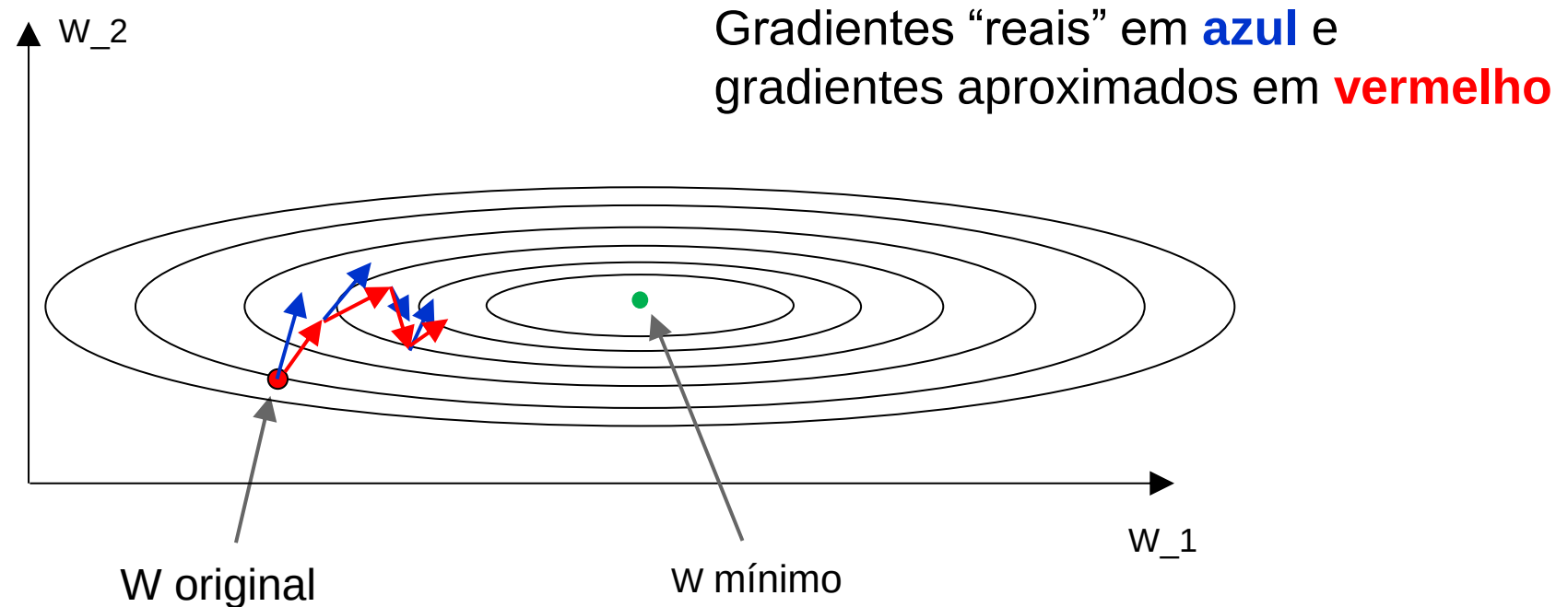
# Gradiente Descendente Estocástico (SGD)

Gradientes apresentam “ruídos” mas ainda são bons para progredir na média em direção ao mínimo



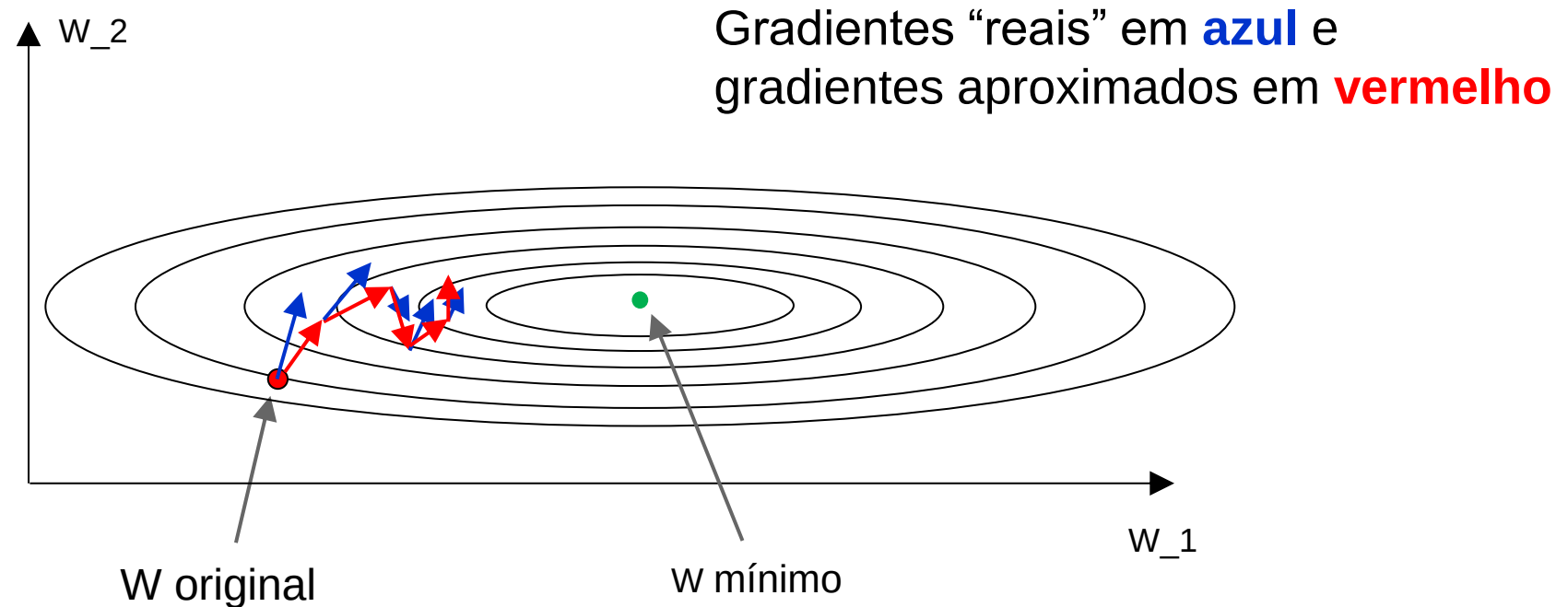
# Gradiente Descendente Estocástico (SGD)

Gradientes apresentam “ruídos” mas ainda são bons para progredir na média em direção ao mínimo



# Gradiente Descendente Estocástico (SGD)

Gradientes apresentam “ruídos” mas ainda são bons para progredir na média em direção ao mínimo



# Gradiente Descendente Estocástico (*SGD*)

Ideia principal: usar apenas uma pequena amostra do conjunto de treinamento para calcular o gradiente em cada passo

# Gradiente Descendente Estocástico (SGD)

Ideia principal: usar apenas uma pequena amostra do conjunto de treinamento para calcular o gradiente em cada passo

```
# Vanilla Minibatch Gradient Descent  
  
while True:  
    data_batch = sample_training_data(data, 256) # sample 256 examples  
    weights_grad = evaluate_gradient(loss_fun, data_batch, weights)  
    weights += - step_size * weights_grad # perform parameter update
```



# Gradiente Descendente Estocástico (SGD)

Ideia principal: usar apenas uma pequena amostra do conjunto de treinamento para calcular o gradiente em cada passo

```
# Vanilla Minibatch Gradient Descent  
  
while True:  
    data_batch = sample_training_data(data, 256) # sample 256 examples  
    weights_grad = evaluate_gradient(loss_fun, data_batch, weights)  
    weights += - step_size * weights_grad # perform parameter update
```

Tamanhos comuns de **minibatches** são amostras de 32, 64, 128, 256, ...  
Por exemplo, na AlexNet utilizou-se lotes com 256 amostras